Layer–Time Geometry

How Language Models Process Information

Agus Sudjianto
H2O.ai

Aijun Zhang
Wells Fargo

A practical companion to
Layer–Time Geometry of Transformer Computation

Preface

Who this book is for

This book is for data scientists, ML engineers, and anyone who wants to understand *what happens inside a transformer* when it processes text. You know what a neural network is. You have used PyTorch or TensorFlow. You may have fine-tuned a language model. But you have probably never asked: what does the internal state of the model *look like*, mathematically, as it moves from layer to layer?

That is the question this book answers. It is a practical, hands-on companion to the research monograph, replacing formal proofs with intuition, code, and visualizations.

Why this book is useful

This book will not teach you to reverse-engineer every circuit in a transformer. Methods like activation patching, probing, and circuit tracing are powerful for explaining model behavior locally—identifying which heads, neurons, or pathways drive a specific output. This book serves a different purpose.

It teaches you how to *measure the global computational regime* of a model: where nontrivial computation concentrates across the full layer–time field, how prompts and models differ internally, how dependency and control are organized, and how to run practical diagnostics for reasoning-related failures, memory use, and steering.

In that sense, the framework sits at a **mesoscopic layer** between black-box evaluation (which tells you *what* the model got wrong) and fine-grained circuit tracing (which tells you *which specific component* is responsible):

Level	Method	Question answered
Black-box evaluation	Benchmarks, red-teaming	Did the model get it right?
Layer–time geometry	**This book**	**Where and how does computation organize?**
Circuit tracing	Activation patching, probing	Which head or neuron is responsible?

Use layer–time geometry to find the interesting regions and regimes; use tracing to zoom in mechanistically.

What you need

- **Linear algebra**: matrix multiplication, eigenvalues, SVD. If you can multiply two matrices and know what an eigenvector is, you are ready.
- **Basic statistics**: mean, variance, PCA, ANOVA (at the "I've seen it in a stats class" level).
- **Python/PyTorch**: enough to load a model and run a forward pass.
- **Calculus**: partial derivatives and gradients. Nothing beyond what you see in machine learning courses.

You do *not* need differential geometry, measure theory, geometric algebra, or operator theory. The research monograph that this book is based on uses all of those. This book translates the key ideas into language you already speak.

How this book works

Every chapter follows the same pattern:

1. **The big question**: what are we trying to understand?
2. **Analogy**: a concrete, everyday comparison.
3. **The idea**: the core concept, in plain language.
4. **The math you need**: only what is necessary, explained step by step.
5. **Python code**: how to compute it yourself.
6. **What we found**: empirical results from real models.
7. **So what?**: why this matters for your work.

Colored boxes throughout the text serve specific purposes:

- **Why Should I Care?** — motivates the topic.
- **Key Idea** — the central takeaway.
- **Analogy** — an intuitive comparison.

- **Try It Yourself** — hands-on exercises.
- **Common Pitfall** — mistakes to avoid.
- **What the Data Shows** — empirical findings.
- **Under the Hood** — optional deeper math (skip on first reading).

Relationship to the research monograph

This book is a companion to *Layer–Time Geometry of Transformer Computation: Kernels, Operators, and Dependency in Language Models* (Sudjianto and Zhang, 2026), which develops the full mathematical framework with proofs, theorems, and formal definitions. When you are ready for the complete treatment, that is where to go. This book gives you the intuition to read it.

The `ltg` library

Every chapter in this book includes runnable code using the `ltg` (Layer-Time Geometry) Python library. This is a student-friendly API that wraps the research-grade `layer_time` package. Install it with pip:

Listing 1: Install the layer-time package

```
pip install layer-time
```

This gives you access to two APIs:

- `import ltg` — the student-friendly API used in this book
- `from layer_time import LayerTimeAnalyzer` — the full research API

The API is designed around three verbs:

Listing 2: The entire ltg workflow in four lines

```python
import ltg

model = ltg.load_model("Qwen/Qwen2.5-7B")      # load once
result = ltg.analyse("What is 2 + 3?", model)  # analyse any prompt
result.summary()                                # see what happened
result.plot_all(prefix="my_analysis")           # generate all plots
```

Every `AnalysisResult` object carries the full geometric decomposition (whitened states, curvature map, polar decomposition, dependency profile) and provides

one-line plotting methods. You can compare prompts, run control experiments, and diagnose failures—all without touching the underlying linear algebra.

Companion Jupyter notebooks

Every chapter has a companion Jupyter notebook in the `tutorials/` directory. These notebooks contain all the code from the chapter—ready to run, with visualisations and exercises. Install the tutorial dependencies with:

Listing 3: Install tutorial extras

```
pip install layer-time[tutorials]
```

Notebook	Chapter
`ch1_opening_the_black_box.ipynb`	Ch. 1: Opening the Black Box
`ch2_whitening.ipynb`	Ch. 2: Standardizing the Representation
`ch3_kernels.ipynb`	Ch. 3: Measuring Similarity: Kernels
`ch4_polar_decomposition.ipynb`	Ch. 4: How Information Flows
`ch5_curvature.ipynb`	Ch. 5: Where the Action Is: Curvature
`ch6_experiments.ipynb`	Ch. 6: Testing What Matters: DOE
`ch7_dependency.ipynb`	Ch. 7: What Each Layer Contributes
`ch8_reasoning_memory_control.ipynb`	Ch. 8: Reasoning, Memory, and Control
`ch9_diagnosing_failures.ipynb`	Ch. 9: Practical Applications

Each notebook is self-contained: load the model once at the top, then work through the cells sequentially. GPU access (CUDA) is recommended but not strictly required—CPU will work for short prompts, just more slowly.

— *The Authors*

Contents

Preface **i**

I What Is Going On Inside? **1**

1 Opening the Black Box **3**
1.1 The question 3
1.2 Hidden states: the model's internal workspace 4
1.3 The four big ideas 5
1.4 What you will be able to do 6
1.5 Hands-on: your first analysis in 60 seconds 6

2 Standardizing the Internal Representation **9**
2.1 The problem with raw hidden states 9
2.2 The solution: PCA-based whitening 10
2.3 Why this matters: metric consistency 10
2.4 Python implementation 11
2.5 Using the ltg library 12
2.6 What we skip on first reading 13

3 Measuring Similarity: Kernels on the Grid **15**
3.1 What is a kernel? 15
3.2 The layer kernel: which layers are similar? 16
3.3 The temporal kernel: which tokens are similar? 16
3.4 What the kernels reveal: three computational phases 16
3.5 Hands-on: visualizing the three phases 17

4 How Information Flows: Operators and Polar Decomposition **19**

4.1 The layer transition operator 19
4.2 Polar decomposition: rotation + stretch 20
4.3 What rotation and stretching mean 21
4.4 Measuring the stretch: eigenvalues of P 21
4.5 Hands-on: exploring the polar decomposition with ltg 22

5 Where the Action Is: Curvature 25
5.1 The idea of curvature on a grid 25
5.2 Computing curvature on the layer–time grid 26
5.3 What curvature tells us 27
5.4 Hands-on: curvature maps with ltg 27

II What Do the Experiments Say? 29

6 Testing What Matters: Experimental Design 31
6.1 The experimental setup 31
6.2 Three models 32
6.3 The results: five key findings 32
6.4 Hands-on: your own mini-DOE 34

7 What Each Layer Contributes: Dependency 37
7.1 The idea 37
7.2 Computing dependency 38
7.3 Dependency profiles 38
7.4 Hands-on: dependency profiles with ltg 39

III The Big Picture 41

8 Reasoning, Memory, and Control 43
8.1 Defining reasoning geometrically 43
8.2 Defining memory 44
8.3 Defining control 44
8.4 Hands-on: running the control experiment yourself 45
8.5 Hands-on: measuring functional memory 47

9 Practical Applications: Diagnosing Model Failures 49
9.1 Application 1: Detecting when a model ignores context 49

9.2 Application 2: Detecting hallucination types 50
9.3 Application 3: Steering model behavior 52

10 What We Learned and What's Next **55**
10.1 The validated findings 55
10.2 What this book did not cover 56
10.3 Open problems for you to work on 56
10.4 The one thing to remember 57

A Introduction to Neural Networks and Transformers **59**
A.1 From linear models to neural networks 59
A.2 The transformer architecture 61
A.3 Types of transformer models 63

B Introduction to GPT and Large Language Models **65**
B.1 The big picture: what GPT does 65
B.2 Stage 1: Tokenization 66
B.3 Stage 2: Token embedding 67
B.4 Stage 3: Positional encoding 68
B.5 Stage 4: Transformer layers (the core computation) 68
B.6 Stage 5: Final layer normalization 72
B.7 Stage 6: Language model head 72
B.8 Stage 7: Decoding 73
B.9 The complete forward pass: a worked example 75
B.10 Training: how GPT learns 76
B.11 Scaling: from GPT-2 to modern LLMs 76
B.12 In-context learning 77
B.13 Why study model internals? 78
B.14 Connecting to the main text 78

C Linear Algebra for Layer–Time Geometry **81**
C.1 Vectors, norms, and inner products 81
C.2 Matrices and linear transformations 82
C.3 Special matrix types 83
C.4 Eigenvalues and eigenvectors 84
C.5 Singular value decomposition (SVD) 84
C.6 Covariance and PCA 85
C.7 Polar decomposition 86
C.8 Condition number and effective rank 86

C.9 Norms for matrices 87
C.10 Gram matrices and kernels 88
C.11 Gradients and the Jacobian 88
C.12 Least squares 89
C.13 Mahalanobis distance 89
C.14 Summary: where each tool is used 89

Glossary **92**

Further Reading **93**

Part I

What Is Going On Inside?

CHAPTER 1

Opening the Black Box

Companion notebook: `tutorials/ch1_opening_the_black_box.ipynb` — run the code from this chapter interactively.

Why Should I Care?

You use language models every day—for code completion, summarisation, question answering. But when a model hallucinates, how do you diagnose the failure? When a model ignores your instructions, where in the computation did things go wrong? To answer these questions, you need to see inside.

1.1 The question

A transformer language model takes a sequence of tokens and processes them through a stack of layers. At each layer, every token has a **hidden state**—a high-dimensional vector that encodes everything the model "knows" about that token at that point in the computation.

The central question of this book is:

What is the geometric structure of these hidden states,
and what does it tell us about the model's computation?

1.2 Hidden states: the model's internal workspace

Analogy

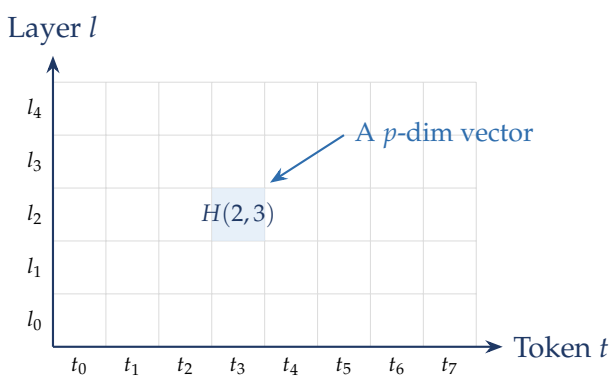
Think of a transformer like a factory assembly line. Raw materials (tokens) enter at one end. At each station (layer), workers (attention heads and feed-forward networks) modify the product. The hidden state at each station is like a detailed spec sheet describing the product's current state—its shape, color, material, intended use. By the time the product reaches the end of the line, the spec sheet contains everything needed to produce the final output.

This book is about reading those spec sheets and understanding what the workers are doing at each station.

Concretely, if a model has L layers and processes T tokens, then at layer l and token position t, the model holds a vector:

$$H(l, t) \in \mathbb{R}^p$$

where p is the model's hidden dimension (e.g., $p = 4096$ for a 7B parameter model). The collection of all these vectors forms a two-dimensional grid:



We call this the **layer–time grid**. "Layer" is the vertical axis (depth in the network). "Time" is the horizontal axis (position in the sequence). Every cell holds a p-dimensional vector.

Key Idea

The hidden states of a transformer form a **field** on a two-dimensional grid. The geometry of this field—how vectors change across layers and across tokens—reveals the model's internal computation.

1.3 The four big ideas

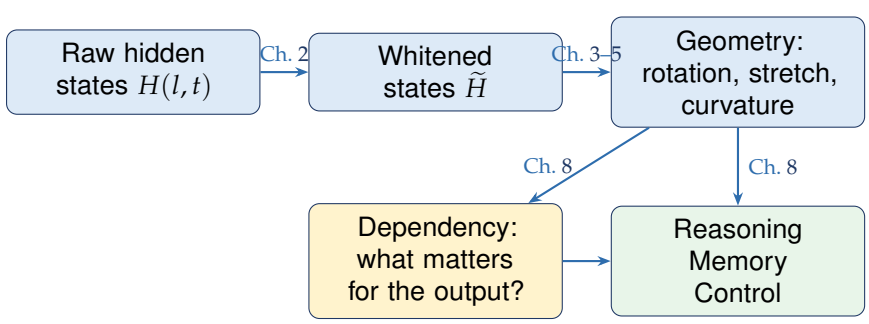
This book develops four main ideas, each building on the last:

1. **Standardize the vectors** (Chapter 2). Raw hidden states have correlated, differently-scaled dimensions. We fix this with PCA-based whitening, like standardizing variables before running a regression.

2. **Measure how vectors change** (Chapters 3–5). We decompose the layer-to-layer transformation into a rotation (direction change) and a stretch (magnitude change), then measure curvature—how much the grid's geometry twists.

3. **Run controlled experiments** (Chapter 6). We use factorial experimental design (DOE) to test which input properties actually affect the geometry: task type? reasoning difficulty? prompt length?

4. **Connect geometry and dependency to reasoning-like computation** (Chapter 8). We add a new observable—*dependency*—that measures how much each layer's hidden state matters for the model's final output. Combining geometry with dependency lets us give practical operational definitions of reasoning, memory, and control.



1.4 What you will be able to do

By the end of this book, you will be able to:

- Extract and whitened hidden states from any Hugging Face transformer.
- Compute the polar decomposition of layer transitions and interpret what rotation and stretching mean.
- Measure curvature and identify where in the network non-trivial computation happens.
- Design and analyze factorial experiments on geometric quantities.
- Compute dependency profiles that show which layers matter most for the model's output.
- Diagnose specific failure modes (hallucination, instruction-ignoring, reasoning-related failures) using geometric and dependency signatures.

1.5 Hands-on: your first analysis in 60 seconds

Listing 1.1: Complete first analysis using the ltg library

```python
import ltg

# Step 1: Load the model (takes ~30 seconds, only do this once)
model = ltg.load_model("Qwen/Qwen2.5-7B")

# Step 2: Analyse a prompt (takes ~10 seconds)
result = ltg.analyse("What is the square root of 144?", model=model)

# Step 3: See the summary
result.summary()
# Output:
#   Tokens: 9  |  Layers: 28  |  Whitened dim: 256
#   Mean curvature: 0.0847
#   Peak layer: 26 (of 27)
#   Final-3-layer share: 42.3%
#   ...

# Step 4: Generate all plots
result.plot_all(prefix="ch1_first")
# Creates: ch1_first_curvature.png, ch1_first_kernel.png,
#          ch1_first_polar.png, ch1_first_dependency.png
```

Try It Yourself

Run the code above. Then try two more prompts:

- A factual question: `"Who wrote Romeo and Juliet?"`
- A reasoning problem: `"If all cats are animals and Whiskers is a cat, what is Whiskers?"`

Compare the three summaries. Which has the highest curvature? Which has the lowest dependency entropy? Do the differences surprise you?

CHAPTER 2

Standardizing the Internal Representation

Companion notebook: `tutorials/ch2_whitening.ipynb` — run the code from this chapter interactively.

Why Should I Care?

If you compare hidden states at different layers without standardizing them first, you are comparing apples and oranges. Whitening is to hidden-state analysis what z-scoring is to statistical analysis: it removes scale differences and correlations so that the geometry you measure is real, not an artefact of the coordinate system.

2.1 The problem with raw hidden states

A transformer's hidden state vector has p dimensions (typically 2048–8192). These dimensions are *not* independent:

- Some dimensions carry much more variance than others.
- Many dimensions are correlated.
- The scale changes from layer to layer.

If you directly compute distances between hidden states at different layers, the distance is dominated by whichever dimensions happen to have the largest variance—

not necessarily the most informative ones.

Analogy

Imagine measuring the similarity between cities using two features: population (in millions) and average temperature (in degrees). Population ranges from 0.01 to 30; temperature ranges from -20 to 45. Without standardisation, Euclidean distance is almost entirely determined by population, and temperature barely matters. The same problem occurs in hidden states, but with thousands of dimensions.

2.2 The solution: PCA-based whitening

The fix is a standard statistical tool: **whitening via PCA**.

Step 1: Center. Subtract the mean hidden state $\bar{H}$ computed across all layers and tokens for a given input.

Step 2: Compute the covariance. The $p \times p$ covariance matrix Σ captures how dimensions co-vary.

Step 3: PCA projection. Take the top k eigenvectors $V_k = [v_1, \ldots, v_k]$ of Σ, with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_k > 0$.

Step 4: Whiten. Define the whitening matrix $W = V_k \operatorname{diag}(\lambda_1^{-1/2}, \ldots, \lambda_k^{-1/2})$ and compute:

$$\widetilde{H}(l,t) = \big(H(l,t) - \bar{H}\big)\, W \quad \in \mathbb{R}^k$$

The result: $\widetilde{H}$ has uncorrelated dimensions with unit variance. We use $k = 256$ throughout—this retains the meaningful variation while making computation tractable.

Key Idea

After whitening, Euclidean distance in the k-dimensional space equals Mahalanobis distance in the original p-dimensional space. This means the geometry we measure respects the model's internal metric, not an arbitrary coordinate choice.

2.3 Why this matters: metric consistency

The core theoretical result (proven formally in the research monograph) is:

Metric Consistency Theorem (informal). After whitening, the covariance of the hidden states is the identity matrix. All subsequent geometric operations—decompositions, distances, curvatures—are invariant to the original coordinate system of the model.

In plain language: whitening lets us compare geometry *across layers, across prompts, and across models* in a principled way. Without it, we would be measuring artefacts.

2.4 Python implementation

Listing 2.1: Whitening hidden states from a Hugging Face model

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def extract_and_whiten(model, tokenizer, text, k=256):
    """Extract whitened hidden states from a transformer."""
    inputs = tokenizer(text, return_tensors="pt").to(model.device)

    with torch.no_grad():
        out = model(**inputs, output_hidden_states=True)

    # Stack all layers: shape (L+1, T, p)
    H = torch.stack(out.hidden_states)     # includes layer 0
    H = H[1:]                               # skip embedding layer
    L, T, p = H.shape

    # Step 1: Centre
    H_flat = H.reshape(-1, p)               # (L*T, p)
    H_mean = H_flat.mean(dim=0)
    H_centred = H_flat - H_mean

    # Step 2-3: PCA via SVD
    U, S, Vt = torch.linalg.svd(H_centred, full_matrices=False)
    V_k = Vt[:k].T                          # (p, k)
    eigenvalues = (S[:k] ** 2) / (L * T - 1)

    # Step 4: Whiten
    W = V_k * (1.0 / eigenvalues.sqrt())    # (p, k)
    H_whitened = H_centred @ W              # (L*T, k)
    H_whitened = H_whitened.reshape(L, T, k)

    return H_whitened  # shape: (L, T, k)
```

2.5 Using the ltg library

The `ltg.analyse()` function performs whitening automatically. But you can inspect the whitened states directly:

Listing 2.2: Inspecting whitened hidden states with ltg

```python
import ltg
import numpy as np

model = ltg.load_model("Qwen/Qwen2.5-7B")
result = ltg.analyse("The capital of France is", model=model)

# Access the whitened hidden states
H_w = result.H_whitened  # shape: (L, T, k)
print(f"Shape: {H_w.shape}")
# e.g., Shape: (28, 7, 256)

# Verify whitening: covariance should be close to identity
H_flat = H_w.reshape(-1, H_w.shape[2])  # (L*T, k)
cov = np.cov(H_flat.T)                   # (k, k)

# Check: diagonal should be ~1, off-diagonal should be ~0
print(f"Mean diagonal:     {np.diag(cov).mean():.4f}")   # ~1.0
print(f"Mean off-diagonal: {np.abs(cov - np.diag(np.diag(cov))).mean
    ():.6f}")  # ~0.0
print(f"Max off-diagonal:  {np.abs(cov - np.diag(np.diag(cov))).max()
    :.4f}")
```

Try It Yourself

Exercise 2.1: Run the code above and verify the covariance is approximately I_k.

Exercise 2.2 (Whitening dimension): Try $k = 32, 64, 128, 256$. For each, compute the curvature profile and check whether the shape changes. At what k does the profile stabilise?

```python
# Hint: use the core library directly for custom k
import layer_time_geometry as core

for k in [32, 64, 128, 256]:
    H_raw = core.extract_hidden_states(model.hf_model, model.
        tokenizer,
                                       "The capital of France is
                                           ", "cuda")
```

```python
    H_np = H_raw[1:].cpu().numpy()
    metric = core.estimate_metric(H_np, n_components=k)
    H_w = core.whiten(H_np, metric)
    Omega = core.curvature(H_w)
    print(f"k={k:3d}: mean_curv={Omega.mean():.4f}, "
          f"peak_layer={Omega.mean(axis=1).argmax()}")
```

2.6 What we skip on first reading

The research monograph proves several additional properties: that whitening is the unique linear transformation achieving metric consistency, that the effective rank of the covariance determines the intrinsic dimensionality, and that the choice of k is stable for $k \geq 128$.

Common Pitfall

Do not skip layer 0. Layer 0 is the raw token embedding before any transformer processing. Including it in the whitening distorts the geometry because it has a fundamentally different distribution. Always start from layer 1. The `ltg` library handles this automatically.

CHAPTER 3

Measuring Similarity: Kernels on the Grid

Companion notebook: `tutorials/ch3_kernels.ipynb` — run the code from this chapter interactively.

Why Should I Care?

Kernels answer the most basic question about the hidden-state grid: which layers are similar to each other? Which token positions are similar? The answers reveal the model's computational phases—setup, processing, and decision-making.

3.1 What is a kernel?

A kernel is just a similarity function. Given two objects a and b, a kernel $K(a, b)$ returns a number indicating how similar they are. You already know several kernels:

- **Dot product**: $K(a, b) = a^\top b$ — the simplest kernel.
- **Cosine similarity**: $K(a, b) = \frac{a^\top b}{\|a\|\|b\|}$ — normalized dot product.
- **RBF kernel**: $K(a, b) = \exp(-\|a - b\|^2/2\sigma^2)$ — similarity decays with distance.

We define two kernels on the layer–time grid.

3.2 The layer kernel: which layers are similar?

For each layer l, collect all T whitened token vectors into a matrix $\widetilde{H}^{(l)} \in \mathbb{R}^{T \times k}$. The **layer kernel** measures how similar two layers are:

$$K_{\text{layer}}(l, l') = \frac{1}{T} \operatorname{tr}\left(\widetilde{H}^{(l)\top} \widetilde{H}^{(l')}\right)$$

This is just the average dot product between corresponding token vectors at two layers.

Analogy

Think of each layer as a photograph of the assembly line taken from above. The layer kernel measures how much two photographs look alike. Early layers look similar to each other (setup phase), late layers look similar to each other (decision phase), and there is often a visible transition in the middle (processing phase).

3.3 The temporal kernel: which tokens are similar?

Similarly, for each token t, collect its representation across all layers: $\widetilde{H}_t \in \mathbb{R}^{L \times k}$. The **temporal kernel** measures:

$$K_{\text{time}}(t, t') = \frac{1}{L} \operatorname{tr}\left(\widetilde{H}_t^\top \widetilde{H}_{t'}\right)$$

3.4 What the kernels reveal: three computational phases

What the Data Shows

When we compute the layer kernel for typical prompts across multiple models (Qwen2.5-7B, DeepSeek-R1-7B, Qwen2.5-32B), we consistently find **three phases**:

1. **Early layers** ($l \lesssim L/4$): High mutual similarity. The model is building up basic representations.
2. **Middle layers** ($L/4 \lesssim l \lesssim 3L/4$): Rapid change. This is where the "work" happens—attention heads route information, representations diverge.
3. **Late layers** ($l \gtrsim 3L/4$): Convergence toward a prediction-ready state. Curvature peaks here.

Listing 3.1: Computing the layer kernel matrix

```
def layer_kernel(H_whitened):
```

```
    """Compute the L x L layer kernel matrix.

    Args:
        H_whitened: tensor of shape (L, T, k)
    Returns:
        K: tensor of shape (L, L)
    """
    L, T, k = H_whitened.shape
    # Reshape to (L, T*k) and compute normalised Gram matrix
    H_flat = H_whitened.reshape(L, T * k)
    K = H_flat @ H_flat.T / T
    return K
```

3.5 Hands-on: visualizing the three phases

Listing 3.2: Visualising the layer kernel to see computational phases

```
import ltg

model = ltg.load_model("Qwen/Qwen2.5-7B")
result = ltg.analyse("Explain why the sky is blue", model=model)

# The layer kernel is already computed
result.plot_layer_kernel(save_path="ch3_kernel.png")

# You will see a block-diagonal structure:
#   - Top-left block (early layers): high similarity
#   - Middle: transition zone
#   - Bottom-right block (late layers): high similarity
# The transition boundary marks where the model shifts
# from "understanding" to "deciding".
```

Try It Yourself

Exercise 3.1: Compare layer kernels for a short prompt (5 tokens) and a long prompt (50+ tokens). Does the three-phase structure hold for both?

Exercise 3.2 (Phase boundary detection): The phase boundary is where consecutive layer similarity drops. Write code to find it:

```
K = result.layer_kernel_matrix
# Consecutive similarity: K[l, l+1] for each l
consec = [K[l, l+1] for l in range(K.shape[0] - 1)]
import numpy as np
```

```
transition = np.argmin(consec)
print(f"Phase transition at layer {transition}")
```

Where does the transition occur relative to the total depth?

CHAPTER 4

How Information Flows: Operators and Polar Decomposition

Companion notebook: `tutorials/ch4_polar_decomposition.ipynb` — run the code from this chapter interactively.

Why Should I Care?

This is the chapter where the framework becomes powerful. We decompose *how* hidden states change from one layer to the next into two fundamental operations: rotation and stretching. This decomposition is the key to understanding what each layer actually does.

4.1 The layer transition operator

Between layer l and layer $l+1$, the hidden states change. We can approximate this change with a linear operator $T^{(l)}$ that best maps $\widetilde{H}^{(l)}$ to $\widetilde{H}^{(l+1)}$:

$$\widetilde{H}^{(l+1)} \approx \widetilde{H}^{(l)}\, T^{(l)}$$

We find $T^{(l)}$ by ordinary least squares—the same linear regression you learned in your first statistics course, just applied to matrices:

$$T^{(l)} = \left(\widetilde{H}^{(l)\top}\widetilde{H}^{(l)}\right)^{-1}\widetilde{H}^{(l)\top}\widetilde{H}^{(l+1)}$$

Analogy

If the hidden states at layer l are the "input data" and the hidden states at layer $l+1$ are the "output data", then $T^{(l)}$ is the best linear predictor. It tells us: what linear transformation best explains how the representation changed?

4.2 Polar decomposition: rotation + stretch

Every square matrix can be decomposed as:

$$T^{(l)} = U^{(l)}\,P^{(l)}$$

where:

- $U^{(l)}$ is an **orthogonal matrix** (rotation/reflection): it changes directions without changing magnitudes.
- $P^{(l)}$ is a **symmetric positive-definite matrix** (stretching): it scales different directions by different amounts.

This is called the **polar decomposition**, and it is analogous to writing a complex number as $z = e^{i\theta} \cdot r$ (angle + magnitude).

Key Idea

Every layer transition in a transformer does two things:

1. **Rotate** the representation (change which directions are active), captured by $U^{(l)}$.
2. **Stretch** the representation (amplify some directions, compress others), captured by $P^{(l)}$.

These are the two fundamental primitives of transformer computation.

4.3 What rotation and stretching mean

Component	What it does	Computational meaning
$U^{(l)}$ (rotation)	Rotates the coordinate axes; preserves all distances	Attention mixing: redistributes information across directions without losing or amplifying anything
$P^{(l)}$ (stretch)	Stretches some directions, compresses others	Feature selection: amplifies important directions, suppresses irrelevant ones

4.4 Measuring the stretch: eigenvalues of P

The eigenvalues $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k$ of $P^{(l)}$ tell us how much each principal direction is stretched:

- $\sigma_i > 1$: direction i is amplified (the model is paying more attention to this direction).
- $\sigma_i < 1$: direction i is compressed (the model is discarding information along this direction).
- $\sigma_i = 1$: direction i is unchanged.

The **condition number** $\kappa = \sigma_1/\sigma_k$ measures how selective the layer is. A high condition number means the layer strongly amplifies some directions while crushing others—it is making a sharp "decision".

The **effective rank** $\text{erank} = \exp(H(\hat{\sigma}))$, where H is the entropy of the normalized eigenvalue distribution, measures how many directions are actively used. Low effective rank means the layer concentrates its computation in a few directions.

Listing 4.1: Polar decomposition of layer transitions

```python
def polar_decompose_layers(H_whitened):
    """Compute polar decomposition T = U @ P for each layer.

    Args:
        H_whitened: (L, T, k) whitened hidden states
```

```
    Returns:
        Us: list of L-1 orthogonal matrices (k, k)
        Ps: list of L-1 symmetric positive-definite matrices (k, k)
    """
    L, T, k = H_whitened.shape
    Us, Ps = [], []

    for l in range(L - 1):
        H_l = H_whitened[l]        # (T, k)
        H_next = H_whitened[l+1]   # (T, k)

        # Least-squares transition operator
        T_l = torch.linalg.lstsq(H_l, H_next).solution  # (k, k)

        # Polar decomposition
        U, S, Vt = torch.linalg.svd(T_l)
        P = Vt.T @ torch.diag(S) @ Vt   # symmetric positive-definite
        U_orth = U @ Vt                   # orthogonal
        Us.append(U_orth)
        Ps.append(P)

    return Us, Ps
```

4.5 Hands-on: exploring the polar decomposition with ltg

The `ltg` library computes the polar decomposition automatically. You can visualize it and inspect individual layers:

Listing 4.2: Polar decomposition analysis with ltg

```
import ltg
import numpy as np

model = ltg.load_model("Qwen/Qwen2.5-7B")
result = ltg.analyse("If x = 3 and y = 2*x + 1, what is y?", model=
    model)

# Plot condition number and effective rank across layers
result.plot_polar(save_path="ch4_polar.png")

# Inspect a specific layer's stretch
layer = 25  # a late layer
P = result.polar_P[layer]             # (k, k) symmetric matrix
sigmas = np.linalg.svd(P, compute_uv=False)  # eigenvalues

```

```python
print(f"Layer {layer} stretch spectrum:")
print(f"  Top 5 eigenvalues:    {sigmas[:5]}")
print(f"  Bottom 5 eigenvalues: {sigmas[-5:]}")
print(f"  Condition number:     {sigmas[0]/sigmas[-1]:.1f}")
print(f"  Effective rank:       {result.effective_ranks[layer]:.1f}")
# High condition number = the layer is being very selective
# Low effective rank = computation concentrates in few directions
```

Try It Yourself

Exercise 4.1 (Rotation vs. stretch magnitude): For each layer, compute:

- $\|U^{(l)} - I\|_F$ (how much rotation deviates from identity)
- $\|P^{(l)} - I\|_F$ (how much stretching deviates from identity)

```python
for l in range(result.n_layers - 1):
    U_dev = np.linalg.norm(result.polar_U[l] - np.eye(result.k),
        'fro')
    P_dev = np.linalg.norm(result.polar_P[l] - np.eye(result.k),
        'fro')
    print(f"Layer {l:2d}: rotation={U_dev:.3f}, stretch={P_dev:.3
        f}")
```

Which layers have more rotation? Which have more stretching? Is there a pattern?

Exercise 4.2 (Compare tasks): Analyze an arithmetic prompt and a factual retrieval prompt. Compare their condition number profiles. Does the model make sharper "decisions" for one task type?

CHAPTER 5

Where the Action Is: Curvature

Companion notebook: `tutorials/ch5_curvature.ipynb` — run the code from this chapter interactively.

Why Should I Care?

Curvature tells you *where* in the layer–time grid the interesting computation happens. Zero curvature means the model could be replaced by a simple linear transformation at that point. High curvature means something genuinely non-trivial is occurring. This is the single most useful diagnostic in the framework.

5.1 The idea of curvature on a grid

In everyday life, curvature measures how much a surface bends. A flat table has zero curvature. A basketball has constant positive curvature. A saddle has negative curvature.

On the layer–time grid, curvature measures something analogous: how much the geometry "twists" when you go around a small loop.

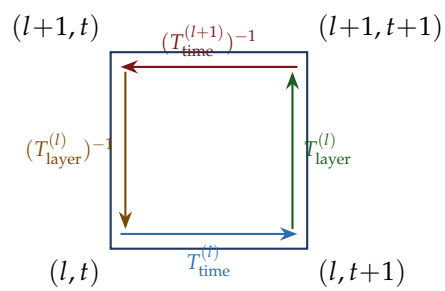
Analogy

Imagine walking around a square block in a city. You make four right turns and end up facing the same direction—zero curvature. Now imagine walking the same square

on the surface of a sphere (like walking a small square near the North Pole). After four right turns, you *do not* face the same direction—the surface's curvature rotated you. The same idea applies to hidden states: go right one token, go up one layer, go left one token, go down one layer. If you end up with a different transformation than the identity, there is curvature.

5.2 Computing curvature on the layer–time grid

Consider a small square ("plaquette") at position (l, t) on the grid:



Going around this loop gives a combined transformation:

$$\Omega(l, t) = T^{(l)}_{\text{time}} \cdot T^{(l)}_{\text{layer}} \cdot \left(T^{(l+1)}_{\text{time}}\right)^{-1} \cdot \left(T^{(l)}_{\text{layer}}\right)^{-1}$$

If the grid were "flat" (i.e., moving right then up gives the same result as moving up then right), then $\Omega(l, t) = I$. The deviation from identity *is* the curvature:

$$\text{curvature at } (l, t) = \|\Omega(l, t) - I\|_F$$

where $\|\cdot\|_F$ is the Frobenius norm (think: sum of squared entries, then square root).

Key Idea

Curvature measures **non-commutativity**: does it matter whether you go "right then up" or "up then right" on the grid? If yes, the computation at that point is genuinely nonlinear and non-trivial.

5.3 What curvature tells us

What the Data Shows

Across all models tested (three architectures, two scales), curvature consistently shows the same pattern:

- **Early layers**: Low curvature. The model is doing simple, nearly-linear processing.
- **Middle layers**: Moderate curvature. Information routing and mixing.
- **Final 3–5 layers**: Curvature spikes sharply. This is where the model commits to its prediction.

This is the most robust finding in the entire framework—it replicates across models, prompts, and tasks.

Common Pitfall

Curvature does not equal reasoning complexity. Early in this research, we conjectured that higher curvature meant harder reasoning. Controlled experiments across three models showed this is **false**: reasoning depth (number of logical steps) has no significant effect on curvature. Curvature measures *non-separable interaction* between layers and tokens—not how "hard" the model is thinking.

5.4 Hands-on: curvature maps with ltg

Listing 5.1: Curvature analysis with ltg

```python
import ltg

model = ltg.load_model("Qwen/Qwen2.5-7B")

# Analyse two prompts of different "difficulty"
r_easy = ltg.analyse("What is 2 + 2?", model=model)
r_hard = ltg.analyse(
    "If f(x) = 3x^2 - 2x + 1, what is f'(x) evaluated at x = 4?",
    model=model
)

# Plot curvature maps for both
r_easy.plot_curvature(save_path="ch5_curv_easy.png")
r_hard.plot_curvature(save_path="ch5_curv_hard.png")

# Compare them side by side
ltg.compare([r_easy, r_hard], save_path="ch5_compare.png")
```

```python
# Print summaries
print("Easy prompt:")
print(f"  Peak curvature layer: {r_easy.curvature_by_layer.argmax()}")
print(f"  Final-3-layer share:  "
      f"{r_easy.curvature_by_layer[-3:].sum()/r_easy.curvature_by_layer.sum():.1%}")
print("Hard prompt:")
print(f"  Peak curvature layer: {r_hard.curvature_by_layer.argmax()}")
print(f"  Final-3-layer share:  "
      f"{r_hard.curvature_by_layer[-3:].sum()/r_hard.curvature_by_layer.sum():.1%}")
# Key finding: the peak location is similar! Curvature does NOT
# increase with problem difficulty.
```

Try It Yourself

Exercise 5.1 (Testing the negative result): Analyze five prompts with increasing reasoning depth (1-step, 2-step, ..., 5-step). Plot their mean curvature. Does it increase? (Spoiler: it should not.)

Exercise 5.2 (Curvature across models): If you have access to a second model (e.g., `Qwen/Qwen2.5-1.5B`), compare curvature profiles on the same prompt. Does the three-phase structure persist at smaller scale?

Part II

What Do the Experiments Say?

CHAPTER 6

Testing What Matters: Experimental Design

Companion notebook: `tutorials/ch6_experiments.ipynb` — run the code from this chapter interactively.

Why Should I Care?

Anyone can compute a metric. The hard part is knowing whether the metric actually tells you something about the input. Experimental design (DOE) is the tool that separates real signal from noise. This chapter uses the same factorial designs you learn in statistics courses—applied to transformer geometry.

6.1 The experimental setup

We designed 320 prompts varying five factors:

Factor	Levels
Cognitive task	Arithmetic, Deductive Reasoning, Causal Reasoning, Linguistic Analysis, Factual Retrieval, Definition
Reasoning depth	Low (1-step), Medium (2-3 steps), High (4+ steps)
Prompt length	Short (<50 tokens), Medium, Long (>150 tokens)
Knowledge domain	Science, History, Everyday, Abstract
Robustness	Original, Paraphrased

For each prompt, we extracted whitened hidden states and computed ∼20 geometric metrics (curvature, condition number, effective rank, dependency measures, etc.). Then we ran ANOVA: which factors significantly affect which metrics?

6.2 Three models

We ran the experiment on three models to check replicability:

Model	Parameters	Layers
Qwen2.5-7B	7 billion	28
DeepSeek-R1-Distill-Qwen-7B	7 billion	28
Qwen2.5-32B	32 billion	64

6.3 The results: five key findings

What the Data Shows

R1: Task type matters, but modestly.

Cognitive task type (arithmetic vs. reasoning vs. retrieval, etc.) is *detectable* in the geometry—certain dependency metrics differ significantly across task types. But the effect sizes are small ($\eta^2 \approx 0.02$–0.05). Task type is a real signal, not the dominant driver.

What the Data Shows

R2: Surface features are not negligible.

We expected that "shallow" features like prompt length and formatting would have no effect on geometry. **Wrong.** Prompt length affects dependency horizons significantly, especially in DeepSeek-R1. Robustness condition drives the largest single effect in Qwen-7B ($\eta^2 = 0.098$ for the global curvature ratio). The geometry responds to both computational content *and* input form.

What the Data Shows

R3: Reasoning depth has no effect on geometry.

This is the cleanest and most important negative result. Across all three models, the factor "reasoning depth" (1-step vs. multi-step problems) *fails to reach significance* on any geometric metric. The one minor exception: condition number in Qwen-7B ($\eta^2 = 0.032$, $p = 0.010$).

What this means: you cannot tell how many reasoning steps a problem requires by looking at the geometry of the hidden states. The number of logical steps does not map to any measurable internal geometric quantity.

What the Data Shows

R4: Curvature concentrates in the final layers.

This finding replicates unanimously across all three models. In Qwen2.5-32B (64 layers), peak curvature is entirely in the final 3 layers. This is the most robust qualitative finding in the study.

What the Data Shows

R5: Dependency tracks task type more closely than curvature.

Dependency-based metrics (how much each layer's state matters for the output) are more sensitive to task type than curvature-based metrics. They are also more stable under paraphrase.

Finding	Qwen-7B	DS-R1	Qwen-32B
Task type detectable	yes	yes	yes
Surface features weak	**no**	**no**	**no**
Curvature = reasoning depth	**no**	**no**	**no**
Curvature in final layers	yes	yes	yes
Dependency stable	yes	yes	yes

6.4 Hands-on: your own mini-DOE

You can replicate a small version of our experiment with `ltg`:

Listing 6.1: Mini factorial experiment with ltg

```
import ltg
import numpy as np
from scipy import stats

model = ltg.load_model("Qwen/Qwen2.5-7B")

# Define prompts by task type
prompts = {
    "arithmetic": [
        "What is 17 * 23?",
        "Calculate 156 / 12",
        "What is 2^10?",
    ],
    "reasoning": [
        "If all dogs are pets and Rex is a dog, is Rex a pet?",
        "If it rains then the ground is wet. The ground is wet. Did it rain?",
        "Alice is taller than Bob. Bob is taller than Carol. Who is shortest?",
    ],
    "retrieval": [
        "What is the capital of Japan?",
        "Who painted the Mona Lisa?",
        "What year did World War II end?",
    ],
}

# Collect metrics by task type
```

```python
results_by_task = {}
for task, texts in prompts.items():
    metrics = []
    for text in texts:
        r = ltg.analyse(text, model=model, compute_dependency=True)
        metrics.append({
            'mean_curv': r.curvature_by_layer.mean(),
            'peak_layer': r.curvature_by_layer.argmax(),
            'dep_entropy': r.dep_entropy,
            'dep_h90': r.dep_horizon_90,
        })
    results_by_task[task] = metrics

# Simple one-way ANOVA on dependency entropy
groups = [
    [m['dep_entropy'] for m in results_by_task[t] if m['dep_entropy']]
    for t in prompts.keys()
]
F, p = stats.f_oneway(*groups)
print(f"ANOVA on dep_entropy: F={F:.3f}, p={p:.3f}")
print("Task type significant?" , "Yes" if p < 0.05 else "No")
```

Try It Yourself

Exercise 6.1: Expand the mini-DOE to 5 prompts per task type and add a fourth task (e.g., "definition"). Run the ANOVA. Which metric shows the strongest task-type effect?

Exercise 6.2 (Paraphrase stability): Take one prompt and create three paraphrases. Analyze all four versions. How similar are their curvature profiles? Compute the pairwise correlation.

CHAPTER 7

What Each Layer Contributes: Dependency

Companion notebook: `tutorials/ch7_dependency.ipynb` — run the code from this chapter interactively.

Why Should I Care?

Geometry tells you *what shape* the hidden states have. Dependency tells you *which parts matter*. A layer might have complex geometry but contribute nothing to the final output. Dependency distinguishes "busy work" from "real work."

7.1 The idea

After a model processes a prompt, it produces a score (log-probability) for the next token. Some layers contributed heavily to this score; others barely mattered. **Dependency** measures this contribution using gradients.

Analogy

In a relay race, some runners gain time and others lose time. The final clock time depends on all runners, but not equally. Dependency is like looking at the split times to figure out who actually made the difference.

7.2 Computing dependency

The dependency of layer l is:

$$D_l = \left\| \frac{\partial s}{\partial \widetilde{H}^{(l)}} \right\|_F$$

where s is the model's score for the next token. This is just the gradient of the output with respect to the hidden state at layer l. Layers where the gradient is large "mattered more" for the output.

Listing 7.1: Computing dependency profiles

```python
def dependency_profile(model, tokenizer, text, k=256):
    """Compute dependency D_l for each layer."""
    inputs = tokenizer(text, return_tensors="pt").to(model.device)

    # Forward pass with gradient tracking
    out = model(**inputs, output_hidden_states=True)
    hidden_states = out.hidden_states[1:]  # skip layer 0

    # Score: log-prob of the predicted next token
    logits = out.logits[0, -1, :]          # last position
    score = logits.max()                    # score of top token

    # Backward pass
    score.backward()

    # Collect gradient norms per layer
    D = []
    for hs in hidden_states:
        if hs.grad is not None:
            D.append(hs.grad.norm().item())
        else:
            D.append(0.0)

    return D  # list of length L
```

Note: the code above is simplified. In practice, you need to enable gradient computation for intermediate hidden states, which requires hooks. See the research monograph's implementation appendix for the full version.

7.3 Dependency profiles

The dependency profile $D_0, D_1, \ldots, D_{L-1}$ reveals which layers drive the output.

Useful summary statistics:

- $D_{\text{total}} = \sum_l D_l$ — total sensitivity to all layers.
- **Dependency horizon** H_α — the layer at which α% of the total dependency is reached. A low H_{90} means the model makes up its mind early.
- **Dependency entropy** $S = -\sum_l \hat{D}_l \log \hat{D}_l$ (where $\hat{D}_l = D_l / D_{\text{total}}$) — how spread out the dependency is. High entropy = many layers contribute; low entropy = a few layers dominate.

Common Pitfall

Length confound. Total dependency D_{total} is strongly correlated with sequence length ($r \approx 0.96$). Longer sequences have more hidden states for the gradient to flow through, mechanically increasing D_{total}. Always use **normalized** metrics (entropy, horizons, concentration ratios) when comparing prompts of different lengths.

7.4 Hands-on: dependency profiles with ltg

Listing 7.2: Computing and visualising dependency profiles

```python
import ltg
import numpy as np

model = ltg.load_model("Qwen/Qwen2.5-7B")

# Analyse with dependency (default: compute_dependency=True)
result = ltg.analyse(
    "The Pythagorean theorem states that a^2 + b^2 = c^2",
    model=model
)

# Plot the dependency profile
result.plot_dependency(save_path="ch7_dependency.png")

# Access the raw profile
D = result.dependency_profile
D_norm = D / D.sum()  # normalise

print(f"Total dependency:          {result.dep_total:.4f}")
print(f"Dependency entropy:        {result.dep_entropy:.3f}")
print(f"Horizon-90 (layer):        {result.dep_horizon_90}")
print(f"Final-3-layer concentration: {result.dep_concentration_final3
    :.1%}")

```

```
# Find the top-3 most important layers
top3 = np.argsort(D_norm)[-3:][::-1]
for l in top3:
    print(f"  Layer {l}: {D_norm[l]:.1%} of total dependency")
```

Try It Yourself

Exercise 7.1 (Length confound): Analyze prompts of lengths 10, 30, and 100 tokens. Plot D_{total} vs. token count. Verify the correlation. Then plot dependency *entropy* vs. token count—is the confound still present?

Exercise 7.2 (Comparing dependency and curvature): For a single prompt, overlay the normalized curvature profile and the normalized dependency profile on the same plot:

```
import matplotlib.pyplot as plt
curv_norm = result.curvature_by_layer / result.curvature_by_layer
    .sum()
dep_norm = result.dependency_profile / result.dependency_profile.
    sum()
# Curvature has L-1 entries, dependency has L entries
plt.plot(curv_norm, label='Curvature (normalised)')
plt.plot(dep_norm, label='Dependency (normalised)')
plt.xlabel('Layer'); plt.legend()
plt.savefig('ch7_curv_vs_dep.png')
```

Do they peak at the same layers? What does it mean when curvature is high but dependency is low?

Part III

The Big Picture

CHAPTER 8

Reasoning, Memory, and Control

Companion notebook: `tutorials/ch8_reasoning_memory_control.ipynb` — run the code from this chapter interactively.

Why Should I Care?

This chapter connects everything. The geometry and dependency tools from earlier chapters combine to give *practical operational definitions* of what it means for a transformer to "reason," "remember," and be "controlled." These are not claims that geometry alone explains reasoning—they are measurable conditions that, when satisfied together, indicate reasoning-like computation that you can detect and test.

8.1 Defining reasoning geometrically

We all have intuitions about what "reasoning" means, but we need a precise definition to measure it. Here is ours:

Key Idea

A transformer is **reasoning** at layer l and token t when three conditions hold simultaneously:

1. **Interaction**: The curvature $\|\Omega(l,t)\|$ is significantly above baseline—the model is doing non-trivial, nonlinear processing.

2. **Propagation**: The metric factor $P^{(l)}$ has high condition number—the model is selectively amplifying some directions.
3. **Dependency**: The dependency $D_{l,t}$ is high—the processing at this point actually matters for the output.

All three must be present. Interaction without dependency is "busy work." Dependency without interaction is just passing information through.

8.2 Defining memory

The framework distinguishes two kinds of memory:

Storage memory: the model's weights encode facts ("Paris is the capital of France"). This is a property of the parameters, not the computation on a specific input.

Functional memory: information from early tokens persists and influences the output at later layers. This *is* a property of the computation, and we measure it as **persistent dependency**:

$$\text{Pers}_\tau = \frac{D_\tau}{D_0}$$

If a token from the beginning of the sequence ($\tau = 0$) still has high dependency in the final layers, the model is "remembering" it functionally.

8.3 Defining control

Control means deliberately changing the model's computation by modifying the geometry. The polar decomposition gives us two levers:

- **Metric control**: clamping or modifying the eigenvalues of $P^{(l)}$ to change what the layer amplifies.
- **Rotation control**: modifying $U^{(l)}$ to change the directions of information flow.

The surprise: metric control dominates

The research monograph originally predicted that combining rotation and metric control ("dual control") would work best, with rotation providing the primary lever. The experiments showed the opposite:

What the Data Shows

Metric control alone is sufficient.

Condition	Dep. total	Dep. entropy
Baseline	∼1.00	∼2.45
Rotation-only	∼0.91 (−9%)	∼2.33
Metric-only	∼0.52 (−48%)	∼1.21
Dual (both)	∼0.52 (−48%)	∼1.21

Metric-only and dual control produce *identical* results. Rotation control adds nothing. The reason: the eigenvalue spectrum of $P^{(l)}$ directly controls how gradients flow, while rotation preserves norms and therefore has little effect on gradient magnitudes.

This is an important lesson: the theory predicted one thing, the data showed another, and we updated the theory. Science works.

8.4 Hands-on: running the control experiment yourself

The `ltg` library provides a one-line control experiment:

Listing 8.1: Running the four-condition control study

```python
import ltg

model = ltg.load_model("Qwen/Qwen2.5-7B")

# Run all four control conditions on a single prompt
results = ltg.control_experiment(
    "Explain why the sky appears blue during the day",
    model=model,
    metric_strength=0.5,     # how aggressively to clamp eigenvalues
    rotation_strength=0.3,   # how much to perturb rotation
)

# Print the results
print(f"{'Condition':15s} {'D_total':>8s} {'H_90':>5s} {'Entropy':>8s}")
print("-" * 40)
for name, r in results.items():
    print(f"{name:15s} {r.dep_total:8.3f} {r.dep_horizon_90:5d} "
          f"{r.dep_entropy:8.3f}")
```

```python

# Visualise
ltg.plot_control(results, save_path="ch8_control.png")
```

Try It Yourself

Exercise 8.1 (Verify metric dominance): Run the control experiment on five different prompts (one per task type). For each, compute the percentage drop in D_{total} for metric-only vs. baseline. Is the ∼45% drop consistent across tasks?

Exercise 8.2 (Strength sweep): Vary `metric_strength` from 0.1 to 0.9 in steps of 0.1. Plot D_{total} vs. strength. Is the relationship linear?

Exercise 8.3 (Detecting reasoning): Implement the three-condition reasoning test. A layer is "reasoning" if all three conditions hold:

```python
import numpy as np

result = ltg.analyse(
    "If all mammals are warm-blooded and whales are mammals, "
    "are whales warm-blooded?",
    model=model
)

for l in range(result.n_layers - 1):
    # Condition 1: Curvature above median
    curv_active = result.curvature_by_layer[l] > np.median(
        result.curvature_by_layer)
    # Condition 2: High condition number (selective stretching)
    stretch_active = result.condition_numbers[l] > np.median(
        result.condition_numbers)
    # Condition 3: High dependency
    if result.dependency_profile is not None:
        dep_norm = result.dependency_profile / result.
            dependency_profile.sum()
        dep_active = dep_norm[l] > 1.0 / result.n_layers  # above
             uniform
    else:
        dep_active = False

    if curv_active and stretch_active and dep_active:
        print(f"  Layer {l}: REASONING "
              f"(curv={result.curvature_by_layer[l]:.4f}, "
              f"kappa={result.condition_numbers[l]:.1f}, "
              f"dep={dep_norm[l]:.3f})")
```

Which layers qualify as "reasoning" layers? Are they concentrated at the end of the network?

8.5 Hands-on: measuring functional memory

Functional memory means information from early tokens persists in late layers. We measure this through the dependency map:

Listing 8.2: Measuring functional memory via dependency persistence

```
import ltg
import layer_time_geometry as core
import numpy as np

model = ltg.load_model("Qwen/Qwen2.5-7B")

text = "Paris is the capital of France. What country is Paris in?"

# Get the full (L, T) dependency map using the core library
H_raw = core.extract_hidden_states(model.hf_model, model.tokenizer,
                                   text, model.device)
H_np = H_raw[1:].cpu().numpy()
metric = core.estimate_metric(H_np, n_components=256)
dep_result = core.compute_dependency_density(
    model.hf_model, model.tokenizer, text, metric, device=model.
        device
)

D_lt = dep_result.D_lt  # shape (L, T)
L, T = D_lt.shape

# Persistence: does the first token's dependency persist to late
    layers?
first_token_dep = D_lt[:, 0]  # dependency of token 0 across all
    layers
last_token_dep = D_lt[:, -1]  # dependency of last token across all
    layers

print("Token 0 ('Paris') dependency by layer (first 5, last 5):")
print(f"  Early: {first_token_dep[:5]}")
print(f"  Late:  {first_token_dep[-5:]}")

# Persistence ratio
persistence = first_token_dep[-3:].mean() / (first_token_dep[:3].mean
    () + 1e-12)
```

```
print(f"  Persistence ratio (late/early): {persistence:.3f}")
# > 0.1 suggests the model "remembers" this token
```

Try It Yourself

Exercise 8.4 (Memory decay curve): For a 20-token prompt, compute the persistence ratio for each token. Plot persistence vs. token position. Do earlier tokens have lower or higher persistence? How does this relate to the attention mechanism?

CHAPTER 9

Practical Applications: Diagnosing Model Failures

Companion notebook: `tutorials/ch9_diagnosing_failures.ipynb` — run the code from this chapter interactively.

Why Should I Care?

This chapter shows why all the theory matters. The geometric framework gives you concrete tools to diagnose and sometimes fix specific model failures that you encounter in practice. Every application includes runnable code.

9.1 Application 1: Detecting when a model ignores context

The problem. You provide a document and ask a question about it. The model answers from parametric memory instead of the provided context.

The geometric signature. When the model uses the context, the dependency profile and curvature profile change compared to the no-context version. When the model ignores the context, the two profiles are nearly identical.

Listing 9.1: Detecting context-ignoring with ltg

```
import ltg

```

```python
model = ltg.load_model("Qwen/Qwen2.5-7B")

# Prompt WITH context
r_with = ltg.analyse(
    "According to the following text, the population of Mars Base "
    "Alpha is 847 people. How many people live on Mars Base Alpha?",
    model=model
)

# Prompt WITHOUT context (same question, no document)
r_without = ltg.analyse(
    "How many people live on Mars Base Alpha?",
    model=model
)

# Detect context ignoring
diagnosis = ltg.detect_context_ignoring(r_with, r_without)
print(f"Context influence score: {diagnosis['context_influence']:.2f}")
print(f"Curvature correlation:  {diagnosis['curvature_correlation']:.3f}")
print(f"Dep. entropy diff:      {diagnosis['dependency_entropy_diff']:.3f}")
print(f"Interpretation:         {diagnosis['interpretation']}")

# Compare visually
ltg.compare([r_with, r_without], save_path="ch9_context.png")
```

Try It Yourself

Exercise 9.1 (Context influence sweep): Create prompts where the context provides:

a) Correct, unique information ("The Zorgon treaty was signed in 2019")
b) Correct, common information ("Water boils at 100°C")
c) Contradictory information ("The capital of France is Berlin")

For each, run the context-ignoring detection. In which case does the model use the context most? In which does it ignore it?

9.2 Application 2: Detecting hallucination types

The problem. A model generates plausible-sounding but incorrect text.

The approach. Different hallucination types produce different geometric signatures. We can build a simple classifier:

Listing 9.2: Geometric hallucination diagnostics

```python
import ltg
import numpy as np

model = ltg.load_model("Qwen/Qwen2.5-7B")

def hallucination_check(text, model):
    """Run geometric diagnostics on a prompt."""
    result = ltg.analyse(text, model=model)
    report = ltg.diagnose(result)

    print(f"Prompt: {text[:50]}...")
    print(f"  Peak curvature layer:  {report.curvature_peak_layer}")
    print(f"  Final-3 curv share:    {report.curvature_final3_share
        :.1%}")
    print(f"  Mean condition number: {report.condition_number_mean:.1
        f}")
    if report.dep_entropy:
        print(f"  Dependency entropy:    {report.dep_entropy:.3f}")
    print(f"  Flags:")
    for flag in report.flags:
        print(f"    - {flag}")
    if not report.flags:
        print(f"    (none)")
    return report

# Test on several prompts
prompts = [
    # Likely correct
    "The Earth orbits the Sun once every 365.25 days.",
    # Likely hallucination (unsupported specificity)
    "The first person to climb K2 without oxygen was "
    "Maria Gonzalez in 1987.",
    # Reasoning that might fail
    "If a train travels at 60 mph for 2.5 hours, "
    "it covers exactly 155 miles.",
]

for p in prompts:
    hallucination_check(p, model)
    print()
```

Hallucination type	Geometric signal	What to check
Unsupported addition	High curvature spike at generation point	`curvature_final3_share` > 0.5
Contradiction	Two dependency peaks (conflicting signals)	Multiple local maxima in D_l
Reasoning error	High curvature + low dependency ("busy but useless")	`low_curvature` flag absent, `shallow_dependency` present

Try It Yourself

Exercise 9.2 (Build a detector): Write a function that takes a prompt and returns a hallucination risk score (0–1) based on the geometric diagnostics. Test it on 10 factual claims (5 true, 5 false). Does the score separate them?

Hint: combine curvature concentration, condition number, and dependency entropy into a simple linear score:

```
def risk_score(result):
    curv_conc = result.curvature_by_layer[-3:].sum() / \
                result.curvature_by_layer.sum()
    kappa = result.condition_numbers.max()
    dep_ent = result.dep_entropy or 0
    # Higher concentration + higher selectivity + lower entropy =
        riskier
    return 0.4 * curv_conc + 0.3 * min(kappa / 100, 1) + \
           0.3 * max(0, 1 - dep_ent / 3)
```

9.3 Application 3: Steering model behavior

The problem. You want to change the model's behavior at inference time without retraining.

The framework view. Every steering method is a special case of:

$$\widetilde{H}^{(l)}_{\text{steered}} = U_\Delta \, \widetilde{H}^{(l)} \, P_\Delta$$

The framework tells you *where* to intervene (layers with both high curvature and dependency), *what* to change (the metric factor P, based on our control study), and *how much* (guided by the eigenvalue spectrum).

Listing 9.3: Identifying intervention points for steering

```python
import ltg
import numpy as np

model = ltg.load_model("Qwen/Qwen2.5-7B")
result = ltg.analyse(
    "Write a positive review of the restaurant",
    model=model
)

# Find layers where BOTH curvature and dependency are above median
curv = result.curvature_by_layer
dep = result.dependency_profile[:len(curv)]  # align lengths
curv_thresh = np.median(curv)
dep_thresh = np.median(dep)

print("Recommended intervention layers:")
for l in range(len(curv)):
    if curv[l] > curv_thresh and dep[l] > dep_thresh:
        print(f"  Layer {l}: curv={curv[l]:.4f}, dep={dep[l]:.4f}, "
              f"kappa={result.condition_numbers[l]:.1f}")

# These are the "decision points" where steering has maximum effect
# Intervening at low-curvature or low-dependency layers is wasted
    effort
```

Try It Yourself

Exercise 9.3 (Steering target selection): For three different prompts (factual, creative, analytical), identify the recommended intervention layers. Are they always in the same relative position (e.g., last 20% of layers)?

Exercise 9.4 (Full diagnostic pipeline): Write a function `full_diagnostic(text, model)` that:

1. Runs `ltg.analyse()`
2. Runs `ltg.diagnose()`
3. Identifies intervention layers
4. Prints a formatted report
5. Saves all plots to a directory

This is your personal "model X-ray" tool.

```python
def full_diagnostic(text, model, output_dir="diagnostic"):
    import os; os.makedirs(output_dir, exist_ok=True)

    result = ltg.analyse(text, model=model)
    report = ltg.diagnose(result)

    print(f"=== DIAGNOSTIC REPORT ===")
    print(f"Prompt: {text[:60]}...")
    result.summary()
    print(f"\nFlags:")
    for f in report.flags:
        print(f"  - {f}")

    # Intervention layers
    curv = result.curvature_by_layer
    dep = result.dependency_profile[:len(curv)]
    hot = [(l, curv[l], dep[l]) for l in range(len(curv))
           if curv[l] > np.median(curv) and dep[l] > np.median(
               dep)]
    print(f"\nIntervention targets: layers {[h[0] for h in hot]}"
        )

    result.plot_all(prefix=f"{output_dir}/diag")
    print(f"\nPlots saved to {output_dir}/")
    return result, report

# Usage:
# full_diagnostic("Your prompt here", model)
```

CHAPTER 10

What We Learned and What's Next

10.1 The validated findings

Here is what the framework establishes with cross-model empirical support:

1. **Three-phase computation.** Transformers consistently show early (setup), middle (processing), and late (decision) computational phases, visible in the layer kernel and curvature profile.

2. **Curvature concentrates in final layers.** The most non-trivial computation happens at the end of the network. This replicates across architectures and scales.

3. **Task type is detectable but not dominant.** The type of cognitive task affects dependency metrics, but with modest effect sizes. The geometry does not cleanly separate "arithmetic" from "reasoning."

4. **Reasoning depth is invisible to geometry.** The number of logical steps in a problem has no measurable geometric signature. This was the most important negative result.

5. **Surface features matter.** Prompt length, formatting, and paraphrasing all affect the geometry—you cannot ignore them as confounds.

6. **Metric control dominates rotation control.** For intervening on model computation, modifying the stretching component is sufficient; changing rotation directions adds nothing measurable.

10.2 What this book did not cover

The research monograph contains much more:

- **Geometric algebra**: a unified language for rotation and stretching using bivectors and rotors.
- **Formal proofs**: metric consistency, orthogonal invariance of dependency, local controllability.
- **Detailed worked examples**: six full-length case studies with algorithms (Anti-Vibe, GRASP, RoRA, SPA, hallucination taxonomy, BLADE).
- **Statistical tests**: formal hypothesis testing for curvature localisation, separability, and representation collapse.
- **Diffusion operators and transport theory**: connections to spectral graph theory and parallel transport.

If this book gave you the intuition, the monograph gives you the full mathematical machinery.

10.3 Open problems for you to work on

These are genuine open questions—any of them could be a research project:

1. **Why is reasoning depth invisible?** Our experiments show that step count does not affect geometry. But *something* must differ internally between 1-step and 5-step problems. What is it? Can a nonlinear probe detect what linear geometry cannot?

2. **Cross-architecture geometry.** We tested three models from two families. Does the three-phase structure hold for fundamentally different architectures (Mamba, RWKV, mixture-of-experts)?

3. **Training dynamics.** How does the curvature profile evolve during training? Do the three phases emerge early or late?

4. **Multimodal models.** Vision-language models process images and text together. What does the layer–time geometry look like when two modalities interact?

5. **Better steering.** Given that metric control dominates, can we design a principled, gradient-free steering method that only modifies eigenvalues?

Try It Yourself

Pick one of the open problems above. Design an experiment to make progress on it. What data would you need? What metrics would you compute? Write a one-page proposal.

10.4 The one thing to remember

If you take away just one idea from this book, let it be this:

Transformer computation has measurable geometric structure.

Hidden states form a field on a layer–time grid.
Each layer transition decomposes into rotation and stretching.
Curvature reveals where non-trivial computation occurs.
Dependency reveals which computation matters.

Geometry alone does not explain reasoning.
But geometry plus dependency gives you a principled
framework for understanding, diagnosing, and controlling
what language models do.

APPENDIX A

Introduction to Neural Networks and Transformers

This appendix provides the background you need to understand what a transformer language model *does*, so that the geometric tools in the main text make sense. If you have taken a machine learning course, you can skim quickly. If not, read carefully—everything in this book builds on these ideas.

A.1 From linear models to neural networks

A **linear model** predicts an output y from an input vector $\mathbf{x}$ using a weighted sum:

$$y = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + b.$$

The weights $\mathbf{w}$ and bias b are learned from data. This is powerful—logistic regression, linear SVM, and ordinary least squares are all linear models—but it can only capture straight-line (hyperplane) decision boundaries.

A **neural network** stacks multiple layers of linear transformations with *nonlinear activation functions* in between:

$$\mathbf{h}^{(1)} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \qquad \mathbf{h}^{(2)} = \sigma(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}), \qquad \ldots$$

where σ is a nonlinear function (ReLU, sigmoid, tanh, etc.). Each intermediate vector $\mathbf{h}^{(l)}$ is called a **hidden state** at layer l.

Key Idea

The nonlinearity is what gives neural networks their power. Without it, stacking layers would reduce to a single linear transformation—no matter how many layers you add.

Analogy

Think of an assembly line. Each station (layer) receives a work-in-progress item (hidden state), transforms it, and passes it to the next station. By the final station, the raw material has become a finished product.

How networks learn: gradient descent

A neural network learns by minimizing a **loss function** $\mathcal{L}$ that measures how far the network's predictions are from the true answers. The key algorithm is **gradient descent**:

1. **Forward pass:** compute the network's output for a batch of inputs.
2. **Compute loss:** compare the output to the correct answer.
3. **Backward pass (backpropagation):** compute $\partial\mathcal{L}/\partial\mathbf{W}^{(l)}$ for every layer.
4. **Update:** nudge each weight in the direction that reduces the loss: $\mathbf{W} \leftarrow \mathbf{W} - \eta\,\partial\mathcal{L}/\partial\mathbf{W}$.

The learning rate η controls the step size. Modern optimizers (Adam, AdaFactor) adapt η per parameter.

Key architectural ideas

Before transformers, several ideas shaped the field:

- **Convolutional neural networks (CNNs):** use local filters that slide across the input. Revolutionized image recognition.
- **Recurrent neural networks (RNNs):** process sequences one step at a time, passing a hidden state forward. Used for language before transformers.
- **Long short-term memory (LSTM):** an RNN variant with gating mechanisms that help information persist across long sequences.

RNNs and LSTMs have a critical limitation: they process tokens *sequentially*, so they are slow to train on long sequences and struggle to relate distant tokens. Transformers solved both problems.

A.2 The transformer architecture

The transformer was introduced by Vaswani et al. (2017) in "Attention Is All You Need." Its two key innovations are **self-attention** and **positional encoding**.

Tokenization

Before a transformer sees text, it is split into **tokens**—subword units from a fixed vocabulary (typically 32k–128k entries). For example:

```
"Understanding transformers" → ["Under", "standing", "
                    transform", "ers"]
```

Each token is mapped to a learned **embedding vector** of dimension p (commonly $p = 768$ to $p = 8192$). This embedding is the input to the first layer.

Self-attention: the core mechanism

Self-attention lets every token look at every other token and decide how much to "pay attention" to it. For each token t, the model computes:

- A **query** $\mathbf{q}_t = \mathbf{W}_Q \mathbf{h}_t$: "What am I looking for?"
- A **key** $\mathbf{k}_t = \mathbf{W}_K \mathbf{h}_t$: "What do I contain?"
- A **value** $\mathbf{v}_t = \mathbf{W}_V \mathbf{h}_t$: "What information do I provide?"

The attention score between tokens i and j is the scaled dot product of query and key:

$$\text{score}(i, j) = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}},$$

where d_k is the dimension of the keys. These scores are passed through a softmax to produce attention weights α_{ij}, and the output for token i is the weighted sum of all values:

$$\text{Attention}(\mathbf{q}_i) = \sum_j \alpha_{ij} \mathbf{v}_j.$$

Analogy

Self-attention is like a meeting where every participant (token) can ask a question (query) and get answers weighted by relevance (key-query similarity) from every other participant. The answers (values) are then combined.

Multi-head attention

Instead of computing one set of $(\mathbf{Q}, \mathbf{K}, \mathbf{V})$, the transformer uses multiple **attention heads** (typically 12–128), each with its own learned projections. Each head can attend to different aspects of the input. The outputs are concatenated and projected:

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, \mathbf{W}_O.$$

Feed-forward networks

After attention, each token's representation passes through a **feed-forward network** (FFN)—two linear layers with a nonlinearity (typically GELU or SiLU):

$$\text{FFN}(\mathbf{h}) = \mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2.$$

The FFN processes each token *independently*—attention handles cross-token communication, FFN handles per-token computation.

Residual connections and layer normalization

Each sub-layer (attention, FFN) is wrapped with:

1. A **residual connection:** $\mathbf{h} \leftarrow \mathbf{h} + \text{SubLayer}(\mathbf{h})$. This lets gradients flow directly through the network, enabling training of very deep models.
2. **Layer normalization:** scales the hidden state to have zero mean and unit variance. Stabilizes training.

Positional encoding

Self-attention is permutation-invariant—it doesn't know the order of tokens. **Positional encodings** inject position information. Modern models use **Rotary Position Embedding (RoPE)**, which encodes relative positions through rotation of query/key vectors.

Putting it all together: one transformer layer

A single transformer layer performs:

$$\mathbf{h}' = \text{LayerNorm}(\mathbf{h} + \text{MultiHead}(\mathbf{h})),$$

$$\mathbf{h}^{(l+1)} = \text{LayerNorm}(\mathbf{h}' + \text{FFN}(\mathbf{h}')).$$

A transformer model stacks L such layers (typically $L = 12$ to $L = 96$).

Key Idea

This is exactly the layer–time grid from Chapter 1. The hidden state $H(l, t)$ at layer l and token t is the output of the l-th transformer layer for token t. The entire framework in this book operates on these hidden states.

A.3 Types of transformer models

Transformers come in three main flavors:

Type	Attention	Examples
Encoder-only	Bidirectional (sees all tokens)	BERT, RoBERTa
Decoder-only	Causal (sees only past tokens)	GPT, LLaMA, Qwen
Encoder-decoder	Both	T5, BART

This book focuses on **decoder-only** models (also called **causal language models**), because they are the architecture behind GPT, LLaMA, Qwen, DeepSeek, and essentially all modern large language models.

APPENDIX B

Introduction to GPT and Large Language Models

This appendix walks through every step of how a GPT-style large language model processes text—from raw characters to generated output. Understanding this pipeline is essential for interpreting the geometric measurements in the main text, because every quantity we compute (curvature, dependency, condition numbers) operates on the intermediate representations produced at each step.

B.1 The big picture: what GPT does

GPT (Generative Pre-trained Transformer) is a **decoder-only** transformer trained on a single objective: **predict the next token**. Given text so far, the model produces a probability distribution over the entire vocabulary for what comes next. That is it. Every capability you see—question answering, translation, code generation, reasoning—emerges from this one objective applied to enough data.

The full pipeline from input text to output text has seven stages:

Stage	What happens
1. Tokenization	Raw text → sequence of integer token IDs
2. Token embedding	Token IDs → dense vectors
3. Position encoding	Inject position information into embeddings
4. Transformer layers	Repeated self-attention + feed-forward processing
5. Final layer norm	Normalize the output of the last layer
6. Language model head	Project hidden states to vocabulary logits
7. Decoding	Convert logits to text (sampling or greedy)

We now explain each stage in detail, using the prompt `"The capital of France is"` as a running example.

B.2 Stage 1: Tokenization

The model does not see characters or words. It sees **tokens**— subword units from a fixed vocabulary, typically 32,000 to 150,000 entries. The vocabulary is constructed during training using algorithms like **Byte-Pair Encoding (BPE)** or **SentencePiece**.

Key Idea

Tokenization determines what the model's "atoms" of language are. Common words become single tokens; rare words are split into subword pieces. This balances vocabulary size with sequence length.

Example. For Qwen2.5-7B, our running example tokenizes as:

```
"The capital of France is" → ["The", " capital", " of", "
      France", " is"] → [785, 6620, 315, 9822, 374]
```

Each token maps to an integer ID from the vocabulary. The result is a sequence of $T = 5$ integers.

How BPE works. Starting from individual bytes (or characters), BPE repeatedly merges the most frequent adjacent pair into a new token. After thousands of merges:

- Common words (`"the"`, `"is"`) become single tokens.
- Rare words (`"bioluminescence"`) are split: `["bi", "ol", "umin", "escence"]`.

- Any text can be tokenized—no out-of-vocabulary problem.

Listing B.1: Tokenization in Python

```python
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-7B")

text = "The capital of France is"
token_ids = tokenizer.encode(text)
tokens = [tokenizer.decode([tid]) for tid in token_ids]

print(f"Token IDs: {token_ids}")   # [785, 6620, 315, 9822, 374]
print(f"Tokens:    {tokens}")       # ['The', ' capital', ' of', '
    France', ' is']
print(f"Vocab size: {tokenizer.vocab_size}")  # 151,665
```

Common Pitfall

Tokenization is not the same as splitting by spaces. The token `" capital"` includes a leading space. Some words split unpredictably. The number of tokens T determines the width of the layer–time grid.

B.3 Stage 2: Token embedding

Each token ID is mapped to a dense vector of dimension p (the **hidden dimension**) via a learned **embedding table** $\mathbf{E} \in \mathbb{R}^{V \times p}$, where V is the vocabulary size:

$$\mathbf{e}_t = \mathbf{E}[x_t] \in \mathbb{R}^p.$$

This is simply a table lookup—row x_t of the matrix $\mathbf{E}$.

For Qwen2.5-7B, $V = 151{,}665$ and $p = 3{,}584$, so the embedding table alone contains $V \times p \approx 544$ million parameters.

Analogy

Think of the embedding table as a dictionary where each word has a 3,584-dimensional "meaning vector" instead of a text definition. Semantically similar tokens ("king" and "queen") have vectors that point in similar directions.

After this stage, our 5 tokens become a matrix of shape $(5, 3584)$—five vectors, one per token.

B.4 Stage 3: Positional encoding

Self-attention (Stage 4) is **permutation-invariant**: it treats the input as a set, not a sequence. Without position information, the model cannot distinguish "dog bites man" from "man bites dog."

Positional encodings inject position information. Modern GPT models use **Rotary Position Embedding (RoPE)**, which works by rotating the query and key vectors in attention:

$$\mathbf{q}'_t = R(\theta, t)\,\mathbf{q}_t, \qquad \mathbf{k}'_t = R(\theta, t)\,\mathbf{k}_t,$$

where $R(\theta, t)$ is a block-diagonal rotation matrix whose angle increases with position t. The key property is that the dot product $\mathbf{q}'^{\top}_i \mathbf{k}'_j$ depends only on the *relative* distance $|i - j|$, not on absolute positions.

Key Idea

RoPE encodes *relative* position through rotation. This connects directly to the rotation component U in the polar decomposition (Chapter 4): some of the rotation we measure is position-dependent rather than content-dependent.

Older models (GPT-2) used **learned absolute positional embeddings**: a separate embedding table $\mathbf{P} \in \mathbb{R}^{T_{\max} \times p}$ added to the token embeddings: $\mathbf{h}^{(0)}_t = \mathbf{e}_t + \mathbf{P}[t]$.

B.5 Stage 4: Transformer layers (the core computation)

This is where the real processing happens. The model stacks L identical layers (28 for Qwen2.5-7B), each consisting of two sub-layers: **self-attention** and a **feed-forward network**.

Key Idea

This is the layer–time grid from Chapter 1. The hidden state $H(l, t)$ at layer l and token t is the output of the l-th transformer layer for token t. Everything in this book operates on these hidden states.

Step 4a: Layer normalization (pre-norm)

Before each sub-layer, the hidden state is normalized:

$$\hat{\mathbf{h}} = \mathrm{RMSNorm}(\mathbf{h}) = \frac{\mathbf{h}}{\mathrm{RMS}(\mathbf{h})} \odot \boldsymbol{\gamma}, \qquad \mathrm{RMS}(\mathbf{h}) = \sqrt{\frac{1}{p}\sum_{i=1}^{p} h_i^2}.$$

The learnable scale $\boldsymbol{\gamma}$ has p parameters. RMSNorm (used in most modern models) omits the mean centering of the original LayerNorm. It stabilizes training by keeping hidden-state magnitudes in a controlled range.

Step 4b: Multi-head self-attention

Self-attention is the mechanism that lets tokens communicate with each other. For each token t, the model computes three vectors from the normalized hidden state $\hat{\mathbf{h}}_t$:

- **Query** $\mathbf{q}_t = \mathbf{W}_Q \hat{\mathbf{h}}_t$: "What am I looking for?"
- **Key** $\mathbf{k}_t = \mathbf{W}_K \hat{\mathbf{h}}_t$: "What do I contain?"
- **Value** $\mathbf{v}_t = \mathbf{W}_V \hat{\mathbf{h}}_t$: "What information do I provide if selected?"

The weight matrices $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{p \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{p \times d_v}$ are learned.

Computing attention scores. The attention score between token i (as query) and token j (as key) is the scaled dot product:

$$a_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}.$$

The $\sqrt{d_k}$ scaling prevents the dot products from becoming too large, which would push the softmax into saturation.

Causal masking. In a GPT model, token t must not attend to future tokens $t+1, t+2, \ldots$. This is enforced by setting $a_{ij} = -\infty$ for all $j > i$ before the softmax:

$$\alpha_{ij} = \frac{\exp(a_{ij})}{\sum_{k \le i} \exp(a_{ik})}, \qquad j \le i.$$

The resulting attention weights α_{ij} form a lower-triangular matrix.

Weighted aggregation. The attention output for token i is the weighted sum of all value vectors it can see:

$$\mathrm{Attn}(\mathbf{q}_i) = \sum_{j \le i} \alpha_{ij}\, \mathbf{v}_j.$$

Multiple heads. Instead of one set of $(\mathbf{Q}, \mathbf{K}, \mathbf{V})$, the model uses H parallel **attention heads** (e.g., $H = 28$ for Qwen2.5-7B), each with its own projections and operating on a slice of the hidden dimension ($d_k = p/H$). Each head can specialize in different types of relationships (syntactic, semantic, positional). The outputs are concatenated and projected:

$$\mathrm{MultiHead}(\mathbf{h}_t) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\, \mathbf{W}_O,$$

where $\mathbf{W}_O \in \mathbb{R}^{p \times p}$ is the output projection.

Analogy

Multi-head attention is like a committee meeting with H sub-committees. Each sub-committee (head) investigates a different aspect of the question. Their reports are concatenated into a single summary.

Grouped-Query Attention (GQA). Modern models like Qwen2.5 and LLaMA-3 use **GQA**: multiple query heads share a single key-value head. For example, Qwen2.5-7B has 28 query heads but only 4 key-value heads (each shared by 7 query heads). This reduces the KV-cache size by $\sim 7\times$ with minimal quality loss.

Step 4c: Residual connection (attention)

The attention output is *added* to the original hidden state:

$$\mathbf{h}'_t = \mathbf{h}_t + \mathrm{MultiHead}(\mathrm{RMSNorm}(\mathbf{h}_t)).$$

This **residual connection** is critical: it creates a "highway" for gradients during training and ensures that each layer makes an *incremental update* rather than a complete replacement.

Key Idea

Residual connections explain why the layer kernel (Chapter 3) shows high similarity between adjacent layers: each layer only adds a small perturbation to the previous

representation. Without residuals, consecutive layers could produce wildly different representations.

Step 4d: Feed-forward network (FFN)

After attention, each token is processed *independently* by a feed-forward network. Modern GPT models use the **SwiGLU** variant:

$$\mathrm{FFN}(\mathbf{h}) = \big(\mathrm{SiLU}(\mathbf{W}_1\mathbf{h}) \odot \mathbf{W}_3\mathbf{h}\big)\,\mathbf{W}_2,$$

where $\mathrm{SiLU}(x) = x \cdot \sigma(x)$ is a smooth activation, $\odot$ is element-wise multiplication, $\mathbf{W}_1, \mathbf{W}_3 \in \mathbb{R}^{p \times d_{\mathrm{ff}}}$, and $\mathbf{W}_2 \in \mathbb{R}^{d_{\mathrm{ff}} \times p}$.

The intermediate dimension d_{ff} is typically $\sim 2.7 \times p$ (e.g., 18,944 for Qwen2.5-7B with $p = 3{,}584$). The FFN accounts for about two-thirds of the model's parameters.

Key Idea

Self-attention handles *cross-token* communication (which tokens should influence each other). The FFN handles *per-token* computation (transforming each token's representation independently). In the geometric framework, the FFN primarily contributes to the **stretching** (metric factor P) in the polar decomposition, because it amplifies some directions and compresses others within each token's representation.

Step 4e: Residual connection (FFN)

The FFN output is again added via a residual connection:

$$\mathbf{h}_t^{(l+1)} = \mathbf{h}'_t + \mathrm{FFN}(\mathrm{RMSNorm}(\mathbf{h}'_t)).$$

The vector $\mathbf{h}_t^{(l+1)}$ is the hidden state at layer $l+1$ for token t—the next row in the layer–time grid.

Summary: one complete layer

Putting it all together, one transformer layer computes:

$$\begin{aligned} \mathbf{h}'_t &= \mathbf{h}_t^{(l)} + \mathrm{MultiHead}\big(\mathrm{RMSNorm}(\mathbf{h}_t^{(l)})\big) && \text{(attention + residual)} \\ \mathbf{h}_t^{(l+1)} &= \mathbf{h}'_t + \mathrm{FFN}\big(\mathrm{RMSNorm}(\mathbf{h}'_t)\big) && \text{(FFN + residual)} \end{aligned}$$

This is repeated L times. For Qwen2.5-7B ($L = 28$), the input embedding passes through 28 layers of this computation, producing a stack of 28 hidden states per token—the columns of the layer–time grid.

Listing B.2: Tracing the shapes through one layer (pseudocode)

```
# Input: h of shape (T, p) = (5, 3584)

# Attention sub-layer
h_norm = rms_norm(h)                      # (5, 3584)
Q = h_norm @ W_Q                          # (5, d_k*H) -> split into H
    heads
K = h_norm @ W_K                          # (5, d_k*H_kv) -> shared KV
    heads
V = h_norm @ W_V                          # (5, d_v*H_kv)
attn_out = multi_head_attention(Q, K, V)  # (5, 3584)
h = h + attn_out                           # residual -> (5, 3584)

# FFN sub-layer
h_norm = rms_norm(h)                       # (5, 3584)
gate = silu(h_norm @ W1)                   # (5, 18944)
up = h_norm @ W3                           # (5, 18944)
ffn_out = (gate * up) @ W2                # (5, 3584)
h = h + ffn_out                            # residual -> (5, 3584)

# Output: h of shape (T, p) = (5, 3584) -- same shape, new values
```

B.6 Stage 5: Final layer normalization

After the last transformer layer, one more RMSNorm is applied:

$$\mathbf{h}_t^{(\text{final})} = \text{RMSNorm}(\mathbf{h}_t^{(L)}).$$

This ensures the representations entering the language model head have consistent scale.

B.7 Stage 6: Language model head

The **language model head** (also called the **unembedding**) projects each token's final hidden state onto the vocabulary:

$$\boldsymbol{\ell}_t = \mathbf{W}_{\text{head}}\, \mathbf{h}_t^{(\text{final})} \in \mathbb{R}^V,$$

where $\mathbf{W}_{\text{head}} \in \mathbb{R}^{V \times p}$. Many models *tie* this matrix to the transposed embedding table: $\mathbf{W}_{\text{head}} = \mathbf{E}^\top$, so the same vectors that define token meanings are also used to predict tokens.

The vector $\boldsymbol{\ell}_t$ is called the **logits**—raw unnormalized scores for each vocabulary entry. A softmax converts them to probabilities:

$$P(x_{t+1} = w \mid x_1, \ldots, x_t) = \frac{\exp(\ell_{t,w})}{\sum_{w'=1}^{V} \exp(\ell_{t,w'})}.$$

Key Idea

The **dependency** $D_l = \|\partial s / \partial \tilde{H}^{(l)}\|$ from Chapter 7 measures how much each layer's hidden state contributes to these logits. High dependency at layer l means that layer l's computation materially affects the next-token prediction.

B.8 Stage 7: Decoding

Given the probability distribution $P(x_{t+1} \mid \ldots)$, we need to select an actual token. This is the **decoding** step.

Greedy decoding

Pick the highest-probability token: $x_{t+1} = \arg\max_w P(w \mid x_1, \ldots, x_t)$. Fast and deterministic, but can produce repetitive, "safe" text.

Sampling with temperature

Scale the logits before softmax: $P_\tau(w) \propto \exp(\ell_w / \tau)$.

- $\tau < 1$: sharper distribution, more "confident" (closer to greedy).
- $\tau = 1$: unmodified probabilities.
- $\tau > 1$: flatter distribution, more "creative" (more randomness).

Top-k sampling

Restrict sampling to the k most likely tokens, setting all other probabilities to zero. For example, top-$k = 50$ means only the 50 highest-probability tokens are considered.

Top-p (nucleus) sampling

Instead of a fixed k, include the smallest set of tokens whose cumulative probability exceeds p (e.g., $p = 0.9$). This adapts the candidate set to the model's confidence: when the model is certain, very few tokens qualify; when uncertain, many do.

Autoregressive generation loop

Generation proceeds one token at a time:

1. **Prompt encoding:** run the full prompt through all L layers. Cache the key and value matrices (**KV-cache**).
2. **Sample next token:** apply the LM head to the last token's final hidden state, decode one token.
3. **Append and continue:** feed the new token through all layers (using the KV-cache to avoid recomputing past tokens).
4. **Repeat** until the model emits an end-of-sequence token or a maximum length is reached.

Listing B.3: Autoregressive generation (simplified)

```python
# Prompt encoding (prefill)
token_ids = tokenizer.encode("The capital of France is")
hidden_states, kv_cache = model.forward(token_ids)

# Generation loop
for step in range(max_new_tokens):
    logits = lm_head(hidden_states[:, -1, :])  # last token only
    probs = softmax(logits / temperature)
    next_token = sample(probs, top_p=0.9)

    if next_token == eos_token_id:
        break

    # Feed just the new token, reusing cached K,V
    hidden_states, kv_cache = model.forward(
        [next_token], kv_cache=kv_cache
    )
    token_ids.append(next_token)

print(tokenizer.decode(token_ids))
# "The capital of France is Paris."
```

Key Idea

The **KV-cache** is why generation cost scales linearly with sequence length rather than quadratically. At each step, only the new token's queries need to attend to all cached keys/values—the past keys and values are not recomputed.

B.9 The complete forward pass: a worked example

Let us trace the entire pipeline for our running example:

1. **Input:** `"The capital of France is"`
2. **Tokenize:** $[785, 6620, 315, 9822, 374]$ — 5 tokens.
3. **Embed:** Look up 5 rows from $\mathbf{E} \to$ matrix of shape $(5, 3584)$.
4. **Position:** Apply RoPE rotations to Q and K at each layer (positions 0–4).
5. **Layer 1:**
   - RMSNorm $\to$ compute Q, K, V $\to$ causal attention $\to$ residual add.
   - RMSNorm $\to$ SwiGLU FFN $\to$ residual add.
   - Output: $(5, 3584)$—the hidden state at layer 1.
6. **Layers 2–28:** repeat the same computation. Each layer reads the previous layer's output and writes a new $(5, 3584)$ matrix.
7. **Final RMSNorm:** normalize the output of layer 28.
8. **LM head:** project the last token's hidden state ($\mathbf{h}_5^{(28)} \in \mathbb{R}^{3584}$) to logits $\in \mathbb{R}^{151665}$.
9. **Softmax:** convert logits to probabilities. The highest-probability token is likely `"Paris"` (token ID for " Paris").
10. **Decode:** output `"Paris"` and optionally continue generating.

The **layer–time grid** from Chapter 1 is the 28×5 matrix of hidden states produced in steps 5–6. The `ltg.analyse()` function extracts these hidden states and computes all geometric quantities on them.

B.10 Training: how GPT learns

The training objective

GPT is trained on **next-token prediction**. Given a sequence $(x_1, \ldots, x_T)$ from the training corpus, the loss is:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(x_t \mid x_1, \ldots, x_{t-1}).$$

Thanks to causal masking, all T predictions are computed in a single forward pass—making training highly parallelizable (unlike RNNs, which must process tokens sequentially).

Pre-training

The model is trained on a massive text corpus (trillions of tokens) using gradient descent with the Adam optimizer. This **pre-training** phase gives the model broad language understanding, factual knowledge, and basic reasoning abilities.

Fine-tuning and alignment

After pre-training, the model is further refined:

1. **Supervised fine-tuning (SFT):** train on curated (instruction, response) pairs to teach the model to follow instructions.
2. **RLHF / DPO:** use human preference data to align the model's outputs with human expectations. **RLHF** (Reinforcement Learning from Human Feedback) trains a reward model and optimizes against it. **DPO** (Direct Preference Optimization) skips the reward model and directly optimizes on preference pairs.

B.11 Scaling: from GPT-2 to modern LLMs

The GPT approach scales remarkably well. Increasing model size, dataset size, and compute produces predictably better models—a relationship described by **scaling laws** (Kaplan et al., 2020; Hoffmann et al., 2022).

Model	Params	Layers	Dim	Heads	Year
GPT-2	1.5B	48	1600	25	2019
GPT-3	175B	96	12288	96	2020
LLaMA-2 7B	7B	32	4096	32	2023
Qwen2.5-7B	7B	28	3584	28	2024
Qwen2.5-32B	32B	64	5120	40	2024
LLaMA-3 70B	70B	80	8192	64	2024

What the Data Shows

In this book, we primarily analyze Qwen2.5-7B (28 layers, 3584 hidden dimensions, 28 attention heads with 4 KV heads) and validate results on DeepSeek-R1-Distill-Qwen-7B and Qwen2.5-32B. The geometric framework applies to *any* decoder-only transformer.

What makes LLMs "large"?

Three things scale together:

- **Parameters:** more layers (L), wider hidden states (p), more attention heads (H). A 7B-parameter model stores $\sim$14 GB of weights in 16-bit precision.
- **Training data:** modern LLMs train on trillions of tokens from the internet, books, code, and curated datasets.
- **Compute:** training GPT-3 required $\sim 3.6 \times 10^{23}$ FLOPs. Modern frontier models require orders of magnitude more.

B.12 In-context learning

A remarkable emergent property of LLMs is **in-context learning**: the ability to perform new tasks from examples in the prompt, without updating any weights.

Listing B.4: In-context learning: the model infers the task from examples

```
Translate English to French:
  sea otter => loutre de mer
  cheese => fromage
  hello =>
```

The model produces "bonjour" without any French-specific fine-tuning. How? The geometric framework suggests that in-context learning modifies the dependency profile: the example tokens acquire high dependency, steering the model's

computation. The curvature profile also shifts, indicating that the model engages different computational pathways when in-context examples are present.

B.13 Why study model internals?

LLMs are powerful but opaque. When a model hallucinates, refuses a valid request, or gives a subtly wrong answer, **we cannot tell why from the output alone**. Understanding the internal computation is essential for:

- **Debugging:** Why did the model ignore the context I provided? (Chapter 9, Section 1.)
- **Trust:** Is the model retrieving a fact or confabulating? (Chapter 9, Section 2.)
- **Steering:** Can I modify the model's behavior at specific layers without retraining? (Chapter 8, Section 3.)
- **Efficiency:** Which layers can be pruned or quantized without losing important computation? (Chapter 7.)
- **Safety:** How do we ensure models behave as intended when deployed in high-stakes settings?

Key Idea

This book gives you concrete, geometric tools for all of these tasks. The hidden states are not a black box—they have measurable structure, and that structure tells you what the model is doing.

B.14 Connecting to the main text

The main chapters of this book analyze hidden states $H(l, t)$—the vectors produced at each layer and token position. Here is how the GPT pipeline connects to each chapter:

GPT pipeline stage	Where it appears in the book
Tokenization (Stage 1)	Determines grid width T
Token embedding (Stage 2)	Layer 0 of the grid (before whitening)
Positional encoding (Stage 3)	Part of rotation component U (Ch. 4)
Self-attention (Stage 4b)	Curvature from cross-token interaction (Ch. 5)
Feed-forward network (Stage 4d)	Stretching factor P in polar decomp. (Ch. 4)
Residual connections (Stage 4c,e)	High layer-kernel similarity (Ch. 3)
LM head and softmax (Stage 6)	Dependency: $\partial s/\partial H$ (Ch. 7)
Causal masking (Stage 4b)	Last token has highest dependency (Ch. 8–9)
Autoregressive decoding (Stage 7)	Generation trajectory analysis
Scaling (layers, width)	Cross-model validation (Ch. 6)

You do not need to memorize every architectural detail. The essential insight is this: the model produces a grid of hidden-state vectors, one per (layer, token) position. These vectors change systematically from layer to layer. The tools in this book measure *how* they change, and what those changes mean for the model's computation.

APPENDIX C

Linear Algebra for Layer–Time Geometry

This appendix collects the linear algebra tools that appear throughout the main text. We assume you have seen vectors and matrices before; the goal here is to give precise definitions and build the intuition you need to follow the geometric framework. Every concept below is used in at least one chapter—cross-references point you to where each idea shows up.

C.1 Vectors, norms, and inner products

A **vector** $\mathbf{h} \in \mathbb{R}^p$ is an ordered list of p real numbers. In this book, every hidden state at position (l, t) on the layer-time grid is such a vector (Chapter 1).

Key Idea

Think of each hidden state as a point in p-dimensional space. Geometric tools—distance, angle, projection—become tools for understanding what the model is doing at each step.

Euclidean norm. The **length** (or ℓ_2 norm) of a vector is

$$\|\mathbf{h}\| \;=\; \sqrt{h_1^2 + h_2^2 + \cdots + h_p^2} \;=\; \sqrt{\mathbf{h}^\top \mathbf{h}}.$$

We use norms throughout to measure how "far" hidden states are from each other or from the origin.

Inner product. The **dot product** (or inner product) of two vectors is

$$\mathbf{a}^\top \mathbf{b} \;=\; \sum_{i=1}^{p} a_i\, b_i.$$

The dot product encodes both magnitude and alignment: $\mathbf{a}^\top \mathbf{b} = \|\mathbf{a}\|\,\|\mathbf{b}\| \cos\theta$, where θ is the angle between the two vectors. When $\mathbf{a}^\top \mathbf{b} = 0$ the vectors are **orthogonal** (perpendicular).

Cosine similarity. Normalizing the dot product gives a scale-invariant similarity measure:

$$\cos\theta \;=\; \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}.$$

The layer kernel in Chapter 3 averages cosine similarities across token positions to compare entire layers.

C.2 Matrices and linear transformations

A **matrix** $A \in \mathbb{R}^{m\times n}$ is a rectangular array of numbers. Multiplying a matrix by a vector, $\mathbf{y} = A\mathbf{x}$, defines a **linear transformation**: it maps every input vector $\mathbf{x} \in \mathbb{R}^n$ to an output $\mathbf{y} \in \mathbb{R}^m$, and it preserves addition and scaling.

In this book, the central matrix is the **layer transition operator** $T^{(l)}$ (Chapter 4), the best linear map from hidden states at layer l to hidden states at layer $l+1$:

$$\mathbf{h}^{(l+1)} \;\approx\; T^{(l)}\,\mathbf{h}^{(l)}.$$

Transpose. The **transpose** $A^\top$ flips rows and columns: $(A^\top)_{ij} = A_{ji}$. For vectors, $\mathbf{h}^\top$ turns a column vector into a row vector, making the dot product $\mathbf{a}^\top \mathbf{b}$ a 1×1 matrix multiplication.

Matrix inverse. If A is square and non-singular, its **inverse** A^{-1} satisfies $A^{-1}A = AA^{-1} = I$ (the identity matrix). The transition operator is computed via least squares, which uses the **pseudoinverse** when an exact inverse does not exist (Chapter 4).

Trace and determinant. The **trace** $\mathrm{tr}(A) = \sum_i A_{ii}$ is the sum of diagonal entries. The **determinant** $\det(A)$ measures the volume scaling factor of the transformation. For the stretch matrix P:

$$\mathrm{tr}(P) = \sum_i \sigma_i, \qquad \det(P) = \prod_i \sigma_i,$$

where σ_i are the eigenvalues. A near-zero determinant signals near-singularity—the layer is collapsing some directions.

C.3 Special matrix types

Several matrix types play starring roles in the geometric framework.

Symmetric matrices. A matrix A is **symmetric** if $A = A^\top$. Symmetric matrices have real eigenvalues and orthogonal eigenvectors (the *spectral theorem*). The covariance matrix Σ and the stretch factor P are both symmetric.

Positive definite and positive semidefinite. A symmetric matrix A is **positive definite** (PD) if $\mathbf{x}^\top A\,\mathbf{x} > 0$ for every non-zero $\mathbf{x}$. It is **positive semidefinite** (PSD) if ≥ 0 replaces > 0.

- PD means all eigenvalues are strictly positive—every direction is stretched, never collapsed to zero.
- The covariance matrix Σ is always PSD.
- The stretch factor P in the polar decomposition is PSD by construction (Chapter 4).

Orthogonal matrices. A square matrix U is **orthogonal** if $U^\top U = I$. Equivalently, its columns form an orthonormal basis. Orthogonal matrices preserve lengths and angles:

$$\|U\mathbf{x}\| = \|\mathbf{x}\| \quad \text{for all } \mathbf{x}.$$

Geometrically, orthogonal transformations are *rotations* (if $\det U = +1$) or *reflections* (if $\det U = -1$). In the polar decomposition $T = UP$, the factor U is orthogonal: it redirects information without changing magnitudes (Chapter 4).

The identity matrix. The $k \times k$ identity matrix I has ones on the diagonal and zeros elsewhere. When $P = I$, the layer performs no stretching; when $\|P - I\|_F$ is large, the layer's stretching deviates significantly from "do nothing" (Chapter 5).

C.4 Eigenvalues and eigenvectors

An **eigenvector** of a square matrix A is a non-zero vector $\mathbf{v}$ that is only scaled (not rotated) by A:

$$A\mathbf{v} = \lambda\,\mathbf{v},$$

where the scalar λ is the corresponding **eigenvalue**.

Analogy

Imagine pushing a door. Most pushes change both the door's direction and speed. An eigenvector is the special direction where the door only speeds up or slows down—it does not turn. The eigenvalue tells you the speed-up factor.

Spectral theorem (for symmetric matrices). If A is $k \times k$ and symmetric, it has k real eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_k$ with corresponding orthonormal eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$. We can write:

$$A = V\Lambda V^\top, \qquad V = [\mathbf{v}_1 \mid \cdots \mid \mathbf{v}_k], \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_k).$$

This is the **eigendecomposition** (or spectral decomposition).

Why it matters for this book.

- The eigenvectors of the covariance matrix Σ are the principal components used in whitening (Chapter 2).
- The eigenvalues of the stretch factor $P^{(l)}$ reveal which directions each layer amplifies or compresses (Chapter 4).
- The eigenvalue distribution of P determines the effective rank and condition number—our key selectivity metrics (Chapter 4).

C.5 Singular value decomposition (SVD)

The eigendecomposition requires a square symmetric matrix. The **singular value decomposition** generalizes to *any* $m \times n$ matrix:

$$A = U_{\mathrm{svd}}\,\Sigma_{\mathrm{svd}}\,V_{\mathrm{svd}}^\top,$$

where:

- $U_{\mathrm{svd}} \in \mathbb{R}^{m\times m}$ is orthogonal (left singular vectors),

- $\Sigma_{\text{svd}} \in \mathbb{R}^{m \times n}$ is diagonal with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$,
- $V_{\text{svd}} \in \mathbb{R}^{n \times n}$ is orthogonal (right singular vectors).

Key Idea

Every linear transformation can be decomposed into three steps: (1) rotate the input ($V_{\text{svd}}^\top$), (2) stretch along coordinate axes (Σ_{svd}), (3) rotate the output (U_{svd}). The singular values tell you how much stretching happens in each direction.

Connection to eigenvalues. For a symmetric PSD matrix A, the singular values equal the eigenvalues. For a general matrix, the singular values of A are the square roots of the eigenvalues of $A^\top A$.

Use in this book. SVD is the computational engine behind two core operations:

1. **PCA/whitening** (Chapter 2): we compute the SVD of the centered hidden-state matrix to extract the top-k principal components.
2. **Polar decomposition** (Chapter 4): we factor the transition operator $T = UP$ by computing $T = U_{\text{svd}} \Sigma_{\text{svd}} V_{\text{svd}}^\top$ and setting $U = U_{\text{svd}} V_{\text{svd}}^\top$ and $P = V_{\text{svd}}\, \Sigma_{\text{svd}}\, V_{\text{svd}}^\top$.

C.6 Covariance and PCA

Given N data points $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^p$ with mean $\bar{\mathbf{x}}$, the **covariance matrix** is

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^{N} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \;\in\; \mathbb{R}^{p \times p}.$$

Entry Σ_{jk} measures how much dimensions j and k co-vary.

Principal Component Analysis (PCA). PCA finds the directions of maximum variance:

1. Center the data: $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$.
2. Compute Σ (or use SVD of the centered data matrix).
3. Take the top-k eigenvectors $V_k = [\mathbf{v}_1, \ldots, \mathbf{v}_k]$ of Σ.
4. Project: $\mathbf{z}_i = V_k^\top \tilde{\mathbf{x}}_i \in \mathbb{R}^k$.

This reduces the dimension from p to k while preserving as much variance as possible.

Whitening. PCA projection followed by rescaling each component to unit variance:

$$W = V_k \operatorname{diag}(\lambda_1^{-1/2}, \ldots, \lambda_k^{-1/2}), \qquad \mathbf{z}_i = W^\top(\mathbf{x}_i - \bar{\mathbf{x}}).$$

After whitening, the covariance of the $\mathbf{z}_i$ is I_k—all directions have unit variance and zero correlation. This is the essential preprocessing step that makes geometric comparisons valid across layers (Chapter 2).

Common Pitfall

Without whitening, distances and angles in the raw hidden-state space are dominated by a few high-variance directions. Two vectors might look similar only because they share a large component along the top principal axis, even though they differ in every other direction. Whitening removes this bias.

C.7 Polar decomposition

Every invertible matrix T can be uniquely factored as

$$T = UP,$$

where U is orthogonal and P is symmetric positive definite. This is the matrix analog of writing a complex number in polar form $z = e^{i\theta} \cdot r$ (Chapter 4).

Analogy

Imagine a block of clay. Applying T to the clay is equivalent to first *stretching* it (P) and then *rotating* it (U). Separating these two actions lets you ask: "how much did the model reshape information?" (P) versus "which direction did the model point it?" (U).

Computing it via SVD. Given $T = U_{\text{svd}}\, \Sigma_{\text{svd}}\, V_{\text{svd}}^\top$:

$$U = U_{\text{svd}}\, V_{\text{svd}}^\top, \qquad P = V_{\text{svd}}\, \Sigma_{\text{svd}}\, V_{\text{svd}}^\top.$$

The eigenvalues of P are the singular values of T.

C.8 Condition number and effective rank

Two scalar summaries of the eigenvalue spectrum of P appear throughout the book.

Condition number.

$$\kappa = \frac{\sigma_1}{\sigma_k},$$

where σ_1 and σ_k are the largest and smallest eigenvalues of P. A large κ means the layer amplifies some directions far more than others—it is making a *selective* transformation. A κ near 1 means all directions are treated roughly equally (Chapter 4).

Effective rank. Let $\hat{\sigma}_i = \sigma_i / \sum_j \sigma_j$ be the normalized eigenvalue distribution. The **effective rank** is:

$$\text{erank} = \exp\Big(-\sum_{i=1}^{k} \hat{\sigma}_i \log \hat{\sigma}_i\Big) = \exp(H(\hat{\sigma})),$$

where H is the Shannon entropy. This measures how many directions are *actively* used:

- If all σ_i are equal, erank $= k$ (full rank).
- If one σ_i dominates, erank ≈ 1 (all computation in a single direction).

Low effective rank means the layer concentrates its computation in a low-dimensional subspace—a fingerprint of focused, selective processing (Chapters 4 and 6).

C.9 Norms for matrices

We use two matrix norms in this book.

Frobenius norm.

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2} = \sqrt{\text{tr}(A^\top A)} = \sqrt{\sum_i \sigma_i^2},$$

where σ_i are the singular values. Think of it as the Euclidean norm applied to the matrix treated as one long vector. The curvature measure in Chapter 5 uses $\|\Omega^{(l,t)} - I\|_F$ to quantify how much the commutator deviates from the identity.

Spectral norm (operator norm).

$$\|A\|_2 = \sigma_1 = \max_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|.$$

This is the largest singular value—the maximum stretching the matrix can apply. The condition number is $\kappa = \|A\|_2 \cdot \|A^{-1}\|_2$.

C.10 Gram matrices and kernels

A **Gram matrix** stores all pairwise inner products of a set of vectors. Given n vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n$:

$$G_{ij} = \mathbf{x}_i^\top \mathbf{x}_j.$$

In Chapter 3, we build two Gram matrices:

- **Layer kernel** $K^{\text{layer}}(i, j)$: average cosine similarity between token vectors at layers i and j. It answers: "how similar are the representations at these two layers?"
- **Temporal kernel** $K^{\text{token}}(s, t)$: average cosine similarity between tokens s and t across all layers. It answers: "how similarly does the model process these two tokens?"

Block-diagonal structure in the layer kernel reveals the three computational phases (embedding, processing, prediction) described in Chapter 3.

C.11 Gradients and the Jacobian

Gradient. For a scalar-valued function $f(\mathbf{x})$, the **gradient** $\nabla_{\mathbf{x}} f \in \mathbb{R}^p$ points in the direction of steepest increase and has magnitude equal to the rate of change in that direction:

$$\nabla_{\mathbf{x}} f = \begin{bmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_p \end{bmatrix}.$$

Jacobian. For a vector-valued function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the **Jacobian** $J \in \mathbb{R}^{m \times n}$ collects all partial derivatives: $J_{ij} = \partial f_i / \partial x_j$.

Use in this book. The dependency measure in Chapter 7 computes

$$D_l = \left\| \frac{\partial s}{\partial H^{(l)}} \right\|_F,$$

where s is a scalar output (e.g., the logit of the predicted token) and $H^{(l)}$ is the hidden-state matrix at layer l. This is the Frobenius norm of a Jacobian: it measures how much the output would change if the hidden states at layer l were perturbed. Large D_l means layer l has high influence on the prediction.

C.12 Least squares

Given an overdetermined system $A\mathbf{x} \approx \mathbf{b}$ (more equations than unknowns), the **least-squares solution** minimizes the residual:

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|^2.$$

When A has full column rank, the closed-form solution is:

$$\hat{\mathbf{x}} = (A^\top A)^{-1} A^\top \mathbf{b}.$$

The matrix $(A^\top A)^{-1} A^\top$ is the **pseudoinverse** of A.

Use in this book. The transition operator is the least-squares fit $T^{(l)} = \arg\min_T \|H^{(l)}T - H^{(l+1)}\|_F^2$—the best linear map from one layer's hidden states to the next (Chapter 4). In code this is computed by `torch.linalg.lstsq`.

C.13 Mahalanobis distance

The **Mahalanobis distance** between a point $\mathbf{x}$ and a distribution with mean $\boldsymbol{\mu}$ and covariance Σ is

$$d_M(\mathbf{x}, \boldsymbol{\mu}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}.$$

It accounts for correlations between dimensions: two points that are close in Euclidean distance may be far apart in Mahalanobis distance if they differ along a low-variance direction.

Key Idea

After PCA whitening, the covariance becomes I, so the Mahalanobis distance *equals* the ordinary Euclidean distance. This is exactly why whitening is so powerful: it turns the complicated, correlation-aware Mahalanobis distance into plain Euclidean distance, which is much easier to compute and interpret (Chapter 2).

C.14 Summary: where each tool is used

Table C.1 maps each linear algebra concept to the chapter(s) where it plays a central role.

Table C.1: Linear algebra concepts and their roles in this book.

Concept	Chapter(s)	Role
Vectors, norms, inner products	1, 3	Hidden states as geometric objects; kernel similarities
Covariance, PCA, whitening	2	Standardize hidden states so geometry is meaningful
SVD	2, 4	Computational engine for PCA and polar decomposition
Eigenvalues, eigenvectors	2, 4, 6	Principal directions (PCA); stretch spectrum (P); DOE metrics
Polar decomposition ($T = UP$)	4, 8, 9	Separate rotation from stretching; metric vs. rotation control
Orthogonal matrices	4, 8	Rotation factor U; rotation control in steering
Condition number, effective rank	4, 6, 9	Selectivity metrics; DOE analysis; failure diagnosis
Frobenius norm	5, 7	Curvature measure; dependency magnitude
Gram matrices, kernels	3	Layer similarity; three-phase structure
Gradient, Jacobian	7	Dependency: output sensitivity to each layer
Least squares, pseudoinverse	4	Fitting the transition operator $T^{(l)}$
Mahalanobis distance	2	Motivation for whitening
Positive definite matrices	2, 4	Covariance Σ; stretch factor P

Glossary

Term	Plain-language definition
Hidden state $H(l,t)$	The model's internal vector for token t at layer l.
Layer–time grid	The 2D grid with layers on one axis and tokens on the other.
Whitening	PCA-based standardization that makes dimensions uncorrelated and unit-variance.
Layer kernel K_{layer}	Similarity between two layers, averaged over tokens.
Transition operator $T^{(l)}$	Best linear map from layer l to layer $l+1$.
Polar decomposition	Writing $T = UP$: rotation U times stretching P.
Rotation $U^{(l)}$	The direction-changing part of the layer transition.
Metric factor $P^{(l)}$	The stretching part; eigenvalues show which directions are amplified or compressed.
Condition number κ	Ratio of largest to smallest eigenvalue of P; measures selectivity.
Effective rank	How many directions are actively used (entropy-based).
Curvature $\Omega(l,t)$	Non-commutativity of going "right then up" vs. "up then right" on the grid. Measures nonlinear interaction.
Dependency D_l	Gradient norm of the output score w.r.t. layer l's hidden state. Measures how much layer l matters for the output.
Dependency horizon H_α	Layer at which α% of total dependency is reached.
Functional memory	Early-token information that persists and influences the output in late layers.
Metric control	Modifying the eigenvalues of $P^{(l)}$ to change the model's computation.

Further Reading

- **The research monograph:** Sudjianto and Zhang, *Layer–Time Geometry of Transformer Computation: Kernels, Operators, and Dependency in Language Models* (2026). The complete mathematical treatment with proofs and algorithms.
- **Linear algebra review:** Strang, *Introduction to Linear Algebra* (6th ed.). Chapters on eigenvalues, SVD, and positive-definite matrices are especially relevant.
- **PCA and statistical methods:** Hastie, Tibshirani, and Friedman, *The Elements of Statistical Learning* (2nd ed.). Chapters 3 and 14.
- **Transformer architecture:** Vaswani et al., "Attention Is All You Need" (2017). The original paper.
- **Mechanistic interpretability:** Elhage et al., "A Mathematical Framework for Transformer Circuits" (2021). Complementary to our geometric approach.
- **Experimental design:** Montgomery, *Design and Analysis of Experiments* (10th ed.). Background on the factorial DOE used in Chapter 6.